University of Bonn
Department of Computer Science
M.Sc. Computer Science

Fraunhofer IAIS
Department of Knowledge Discovery

Master's Thesis

Evaluating Foundation Models using Keyphrase Analysis without Ground Truth

Submitted by
BEHNAM FANITABASI

Supervisor 1: Prof. Dr. Lucie Flek
Supervisor 2: Dr. Jörg Kindermann
Advisor: Vahid Sadiri Javadi

June 17, 2024

DECLARATION

Hereby, I, Behnam Fanitabasi (Student of M.Sc. Computer Science at the University of Bonn, matriculation number: 3479208), declare that I have composed the present Master´s thesis independently on my own and without any resources other than the ones indicated. The present Master´s thesis has not been used as an examination paper, neither in its current form nor in a similar form, in any other course or examination.

Bonn, June 17, 2024

BEHNAM FANITABASI

© 2024 - BEHNAM FANITABASI
All rights reserved.

ACKNOWLEDGEMENTS

This work would not have been possible without the support and guidance of many individuals.

First and foremost, I would like to express my deepest gratitude to my supervisors, **Dr. Lucie Flek** and **Dr. Jörg Kindermann**, and my advisor, **Vahid Sadiri Javadi**. Your invaluable guidance, constructive feedback, and constant encouragement have been pivotal throughout this journey. I am deeply grateful for your patience and the knowledge you have shared with me.

I owe a profound debt of gratitude to my mother, **Zohreh**. She is the cornerstone of my life, the unwavering support that has enabled me to pursue my dreams. Her sacrifices, love, and encouragement are the reasons I am here today. I cannot thank her enough for everything she has done for me. To my father, **Farzin**, whose memory I always carry with me, I know he is proud and happy to witness my achievement from above.

I would also like to thank my brother, **Farzam**, and my sister-in-law, **Vivian**. Your help, motivation, and willingness to listen to my endless nags have been a constant source of strength. As you always said, you have been just one phone call away. Special thanks to their cat, **Niloo**, whose playful antics and joy have provided me with much-needed moments of happiness and stress relief.

To my grandmother, **Taiyebeh**, whom I affectionately call Mamani, your boundless love and warmth have always been a source of comfort and inspiration. I am also deeply grateful to my uncles, **Mehrdad**, **Masoud**, and **Farzad**. Your beliefs in me have been a guiding light, and your presence in my life has been invaluable.

Finally, I want to express my heartfelt thanks to my partner, **Mahnaz**. Your support, patience, and love have been the bedrock of my strength throughout this journey. From the very first day I arrived in Germany to this very moment, you have stood by my side through the highs, lows, and many panics. Without you, this journey would not have been possible. Thank you for being my rock and for making every step of this adventure worthwhile.

To everyone who has contributed to this thesis, whether through direct guidance, emotional support, or simply by being there, I extend my deepest thanks. Your contributions have been invaluable, and I am forever grateful.

This thesis is dedicated to...

ABSTRACT

Neural natural language models are drawing more and more attention every day as they aim to give computers the ability to interpret human words and documents. Yet as there are more and more models available, it gets more challenging to select the optimal one for a certain task. This thesis focuses on the evaluation of neural language models, particularly Transformer-based models, in the absence of ground truth data. Traditional evaluation methods relying on downstream tasks are often impractical due to the need for costly and time-consuming human-annotated ground truth data. To overcome this limitation, an evaluation method leveraging the embedding space is proposed. The method utilizes cosine similarity metrics to construct Nearest Neighbor Graphs (KNN) from embeddings generated by different language models. By conducting keyphrase analysis and frequency distribution calculations, the method aims to differentiate the performance of various models and datasets across different domains. The report provides detailed methodology, experimental configurations, and results, showcasing the effectiveness of the proposed evaluation strategy in assessing the performance of neural language models across varied inputs. Moreover, the framework's capability in detecting Out-of-Distribution (OOD) datasets is examined, thus underscoring its utility in model assessment and dataset categorization. Finally, the potential features identified in this thesis and their implications for future research are discussed.

CONTENTS

1 Introduction 1
2 Related Work 3
  2.1 Evaluating Foundation Models 3
  2.2 Out-of-Distribution Detection 4
3 Theoretical Foundations 5
  3.1 Neural Natural Language Models and Vector Embedding 5
  3.2 Tokenizer 5
  3.3 Transformer-based Language Models 6
  3.4 Perplexity 8
  3.5 Out-of-Distribution Data 8
  3.6 Term Frequency–Inverse Document Frequency 9
  3.7 Nearest Neighbor and Recursive Nearest Neighbor Graph 9
  3.8 Key-phrase Extraction 11
4 Methodology 15
  4.1 Flow Diagram 15
  4.2 Data Preprocessing 16
    4.2.1 Tokenizer 16
    4.2.2 Embeddings Calculation 16
    4.2.3 Keyphrase Extraction and Recursive KNN Graph Creation 16
  4.3 Tokenizer Training/Fine-tuning 17
  4.4 Language Model Training and Embeddings Calculation 18
  4.5 Keyphrase Extraction 19
  4.6 Keyphrase Filtering 20
  4.7 Document Filtering 20
  4.8 Recursive KNN Graph Creation 20
  4.9 Keyphrase Analysis 21
  4.10 Out-of-Distribution Selection 21
5 Data 23
  5.1 AmazonCats-13k 23
  5.2 MedQuad 23
  5.3 FEVER 24
  5.4 Emotional-Reddit 25
  5.5 xnli and x_stance 25
  5.6 General Properties of the Datasets 26
6 Experiments 27
  6.1 Out-of-Distribution Detection 27
  6.2 model training and fine-tuning 28

6.3 Keyphrase Extraction and Filtering 28
6.4 Configurations for Keyphrase Analysis 29
6.4.1 AmazonCats-13k 30
6.4.2 Additional Datasets 30
7 Results 33
7.1 Different Model Variants on the Same Dataset 33
7.1.1 AmazonCats 33
7.1.2 Extra Experiments 42
7.2 Different Datasets on the same model 44
8 Conclusion 49
8.1 Limitation 50
8.2 Future Work 50
Bibliography 53
A Appendix 61

LIST OF FIGURES

Figure 2 Transformer Architecture [50] 7
Figure 3 Example of a node's environment in a Recursive KNN graph with K=4 and nHops=2 10
Figure 4 Keyphrase Analysis Workflow 15
Figure 5 The black-box model 17
Figure 6 distribution of document length 24
Figure 7 Checkpoints' scores on the evaluation set 33
Figure 8 Keyphrase distributions for different subsets of the AmazonCats dataset 34
Figure 9 Frequency distributions of keyphrases across different checkpoints 35
Figure 10 distributions of 0.2 most and least frequent keyphrases across different checkpoints for K=4 & nHops=4 35
Figure 11 Difference in the frequencies of all keyphrases with K=4 and nHops=4 37
Figure 12 Difference in the frequencies of keyphrases with K=4 and nHops=4 and cut-off=200 38
Figure 13 Difference in the frequencies of keyphrases with K=3 and nHops=3 and cut-off=200 39
Figure 14 Difference in the frequencies of keyphrases with K=3 and nHops=4 and cut-off=200 40
Figure 15 Difference in the frequencies of keyphrases with K=5 and nHops=5 and cut-off=500 41
Figure 16 Distributions of xnli dataset 42
Figure 17 Distributions of x_stance dataset 43
Figure 18 Properties of xnli's recursive KNN graph 43
Figure 19 Frequency distributions of different datasets 45
Figure 20 Frequency distributions of different dataset 46
Figure 21 Weighted frequency distributions of keyphrases for different datasets 47
Figure 22 Difference in the frequencies of keyphrases with K=5 and nHops=5 and cut-off=200 62
Figure 23 Properties of x_stance's recursive KNN graph 63

LIST OF TABLES

Table 1 General properties of the datasets 26
Table 2 The obtained fertility values for datasets from various domains 27
Table 3 The number of keyphrases before and after filtering 29
Table 4 Model's performance on different datasets 46

1

INTRODUCTION

Natural language processing (NLP) is a branch of computer science, specifically artificial intelligence, which employs various techniques to help computers comprehend texts and speeches as people do. Neural language models such as Transformer-based models are an approach that represents words and texts as vector embeddings in order to use the computing power of computers for various tasks involving datasets of words and texts, such as clustering documents according to their topics. Transformer-based [50] and foundation models are often trained on enormous amounts of text data using unsupervised methods. The performance of the machine learning (ML) model trained with these embeddings for a specific task may alter if multiple embeddings of the same dataset obtained by different language models are used. Furthermore, when a language model is exposed to inputs that are significantly different from the data it has seen during training, it may encounter difficulty in generating outputs that are precise or meaningful because it lacks the necessary context and patterns to handle these new inputs effectively. An Out-of-Distribution dataset could contain uncommon words, phrases, topics, or language styles that the model has not encountered in its training data. For example, if a model has only been trained on Wikipedia documents, it might struggle with advertising documents such as Amazon descriptions. Identifying Out-of-Distribution inputs and datasets with varying domains when a model encounters them is crucial since it makes language models safer to use in practical applications. It can prevent the model from generating overly confident, misleading, or nonsensical responses when it encounters unfamiliar input. Thus, it is important to select the model that performs the best on the present dataset.

Traditional methods for evaluating language models typically rely on downstream tasks, such as question answering, code generation, text generation, and text classification. These evaluations often depend on human-annotated ground truth data, which can be costly and time-consuming to acquire. Moreover, due to the nature of these datasets, attempting to obtain different embeddings and train ML models on them to choose the best one is memory and time insufficient. Evaluating the quality of a language model becomes challenging in the absence of a specific downstream task. As a result, alternative evaluation strategies have gained attention, aiming to bypass the need for ground truth data.

The solution is to develop an evaluation method for neural language models based on the embedding space. In previous researches, cosine similarity was employed as a metric to measure the similarity between different embeddings and construct Nearest Neighbor Graphs (KNN), which yielded promising outcomes. This enabled the extraction of various features from the embeddings, facilitating the distinction and analysis of embeddings produced by different models. Further details regarding this *Methodology* and the *Results* are discussed in-depth in the corresponding sections. Thus, in this thesis, an evaluation method is developed and evaluated using KNN graphs and Keyphrase Analysis. A Longformer (Transformer-Based) [5] model is fine-tuned and trained on a chosen dataset and utilized to get the embeddings of different datasets. Using the selected dataset as well as datasets from other domains, the distribution of the frequency of the keyphrases retrieved from the dataset is calculated from the KNN graphs in order to study and evaluate the model's performance and the quality of the generated embeddings. To analyze and identify patterns, the distributions generated by different models are compared with their respective performances. Additionally, the distributions of different datasets are compared to assess the effectiveness of keyphrase analysis methods when dealing with datasets from diverse domains. The *Methodology* section of this report provides an explanation of this approach. The configurations, as well as the results of the experiments and comparisons, are detailed in the *Experiments* and *Results* sections.

2

RELATED WORK

2.1 EVALUATING FOUNDATION MODELS

Several methodologies have emerged to assess the performance and robustness of foundation models in scenarios where ground truth data may be absent or difficult to obtain.

One approach explores intrinsic evaluation metrics, focusing on properties desirable in language models like fluency, coherence, and grammatical correctness. A widely used example is BLEU [34], originally designed for machine translation evaluation. BLEU calculates the similarity between the generated text and a set of reference sentences, but it can struggle to capture real-world usefulness or factual accuracy. Other intrinsic metrics attempt to assess these aspects, but they often face limitations in comprehensively evaluating complex language models' outputs.

Another line of work investigates relative evaluation techniques. Here, the focus is on comparing the performance of different language models on a specific task. This can be achieved through human evaluation where judges compare outputs from multiple models, or through automated methods that rely on tasks designed to expose weaknesses in one model compared to another. For instance, Dhurandhar et al. [17] propose a triplet ranking method where groups of three models assess each other's outputs. If designed effectively, this method can iteratively rank a larger pool of large language models without needing pre-defined correct answers.

Recent research delves into leveraging the capabilities of multiple language models to achieve evaluation without ground truth. Ströbel et al. [45] explore this concept in the context of Handwritten Text Recognition (HTR) models. Their work investigates methods for evaluating the quality of recognized text when ground truth data (i.e., the original handwritten text) is unavailable. This is particularly relevant for historical documents or large datasets where manual annotation is impractical. One approach involves using standard language models (SLMs) to assess the plausibility of the generated text. More recent advancements utilize masked language models (MLMs) which have shown promising results in evaluating HTR models without ground truth.

In addition to the above-mentioned works, a recent survey paper by Chang et al. [11] provides a comprehensive overview of various evaluation methods for large language models, covering task-level, societal impact including fairness and bias, and reasoning capabilities. This survey is a valuable resource for understanding the current landscape of LLM evaluation and it highlights the need for multifaceted evaluation considering various aspects beyond just task-level performance.

2.2 OUT-OF-DISTRIBUTION DETECTION

Foundation models are trained on massive datasets, but their performance can degrade significantly when encountering data outside the training distribution. Detecting out-of-distribution (OOD) data is crucial for ensuring the reliability of language models in real-world applications.

One approach investigates leveraging the inherent properties of transformer-based models for OOD detection. Pollano et al. [37] explore this using topological features extracted from the internal activations of a transformer model. The idea is that these features exhibit distinct patterns when the model encounters OOD data compared to in-distribution data. This method offers a data-driven approach for OOD detection without requiring additional training data.

Another line of work focuses on enhancing sentence embeddings to improve OOD detection capabilities. Chen et al. [13] propose a method for constructing "holistic sentence embeddings" that capture both semantic and syntactic information from the sentence. These embeddings are then used to distinguish between in-distribution and OOD data points. This approach aims to create more robust representations that can effectively identify OOD data, even for complex and nuanced examples.

These advancements in OOD detection are crucial for ensuring the responsible deployment of foundation models in real-world scenarios. By identifying data points that fall outside the model's training domain, one can mitigate potential errors and improve the overall reliability of language models' applications.

However, it is important to note that the approach proposed in this thesis for evaluating foundation models without ground truth is novel and not yet a mainstream technique. While there is much ongoing research in this area, as evidenced by the related works mentioned above, a method directly comparable to this work might not have been published yet, which highlights the potential significance of this work in contributing to the advancement of evaluation and OOD strategies.

3

THEORETICAL FOUNDATIONS

3.1 NEURAL NATURAL LANGUAGE MODELS AND VECTOR EMBEDDING

Natural Language Processing (NLP) is a branch of artificial intelligence (AI) concerned with the interaction between computers and human (natural) languages. It involves the development of algorithms and models that enable computers to understand, interpret, and generate human language in a way that is both meaningful and contextually relevant. Neural natural language models are a type of machine learning model used for natural language processing tasks. These models are based on neural networks and are used to represent words or sentences as dense, high-dimensional vectors which are called word embeddings and sentence embeddings. Word embeddings are vector representations of words that capture the semantic and syntactic relationships between them and sentence embeddings are built by combining the word embeddings. Similar words and texts should be clustered together in the embedding space in order to achieve the purpose of these embeddings. Word2Vec [31] and Glove [36] are examples of famous embedding algorithms. These embeddings can be used in tasks such as question answering, text classification, and information retrieval. For instance, ChatGPT [12] is a chatbot developed by OpenAI and was trained on GPT 3.5 [23] which is a generative language model that uses a transformer architecture [50] to generate high-quality natural language text.

3.2 TOKENIZER

In natural language processing, a tokenizer is a fundamental component responsible for breaking down a given text into smaller units known as tokens which can be words, subwords, or characters, depending on the chosen level of granularity. By simplifying the language, this approach makes it easier for machines to comprehend and process. In many NLP tasks, such as named entity recognition, sentiment analysis, and machine translation, tokenizers are crucial. Different tokenization approaches, such as word-level, subword, and

character-level tokenization, meet specific task requirements. Within the realm of transformer models, like BERT [15] and GPT [38], specialized tokenization is required due to the unique architecture, which utilizes self-attention mechanisms. Transformer tokenization involves converting input text into tokens suitable for the input layer of the model. This often includes special tokens like [CLS], [SEP], and [MASK].

The fertility of a tokenizer refers to the average number of subwords produced per tokenized word [41]. It is a measure of how well the selected tokenizer works with various corpora while converting them into tokens. Fertility can be defined as the ratio of the total number of subtokens generated by the tokenizer to the total number of words in the original text. Mathematically, it can be expressed as:

$$Fertility = \frac{\text{Total Number of Subtokens}}{\text{Total Number of words in Original Text}}$$

Moreover, fertility can be effective in determining how a given text differs from the texts which the chosen tokenizer was pretrained on as a minimum fertility of 1 means that the tokenizer"s vocabulary includes every single word in the given text, and higher fertility indicates the conversion of the words into subtokens.

3.3 TRANSFORMER-BASED LANGUAGE MODELS

Natural language processing has advanced dramatically with transformer-based language [50] models, which have changed the field of AI-driven text generation and understanding. Transformer-based language models leverage the Transformer architecture in figure 2, which relies on self-attention mechanisms to capture contextual relationships within input sequences. Unlike traditional recurrent neural networks (RNNs) or convolutional neural networks (CNNs), Transformers process entire sequences simultaneously, making them highly parallelizable and efficient.

- **Self-Attention Mechanism:** The backbone component of the Transformer architecture, self-attention enables each token in the input sequence to attend to all other tokens, capturing long-range dependencies and contextual information effectively.
- **Positional Encoding:** Since Transformers do not inherently possess sequential information like RNNs, positional encoding is added to the input embeddings to provide information about the position of tokens within the sequence.
- **Feedforward Neural Networks:** Transformer layers typically include feedforward neural networks to process the information obtained from self-attention layers.

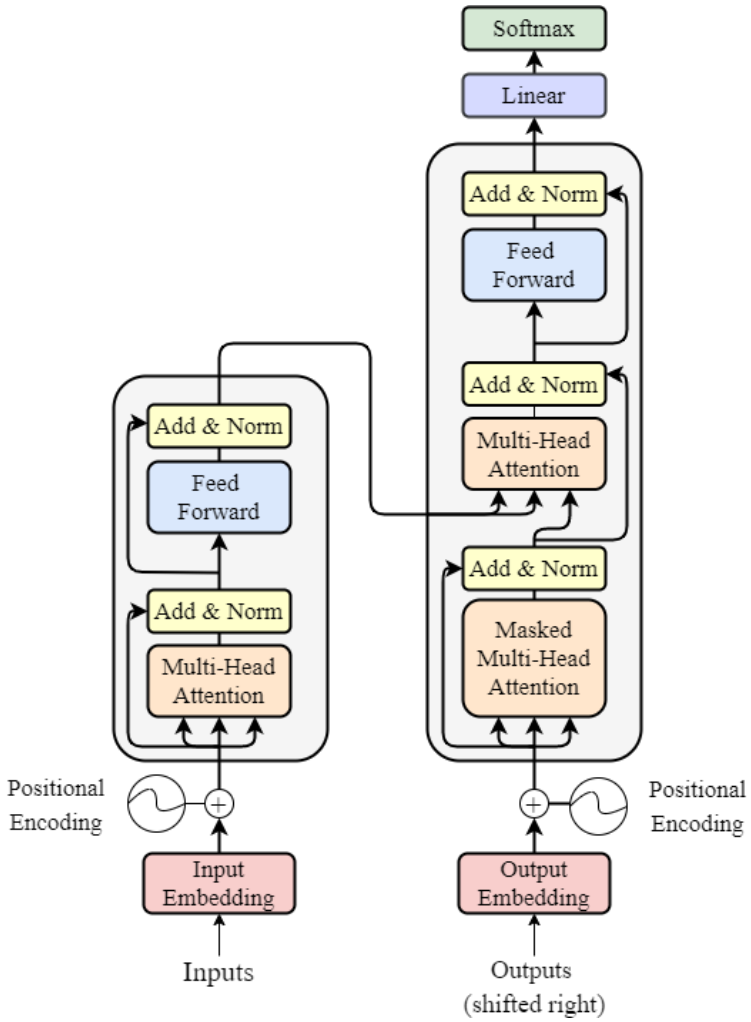


Figure 2: *Transformer Architecture [50]*

Transformer-based language models are often pretrained on large corpora of text using unsupervised learning objectives, such as language modeling or masked language modeling (MLM). This pretraining phase allows the model to learn rich representations of language, capturing syntactic, semantic, and contextual nuances. After pre-training, Transformer-based language models can be fine-tuned on specific downstream tasks with labeled data. When a model is fine-tuned, its parameters are adjusted to better fit the specific requirements of the target task, which could be machine translation, sentiment analysis, or text classification.

BERT (Bidirectional Encoder Representations from Transformers) [15] is a landmark Transformer-based language model that revolutionized the field of natural language processing. Unlike previous models that processed text in a left-to-right or right-to-left manner, BERT employs a bidirectional approach, allowing it to capture context from both directions simultaneously. BERT was pretrained on

BookCorpus, a dataset consisting of 11,038 unpublished books [51], and English Wikipedia (excluding lists, tables, and headers) [21] in a self-supervised fashion with two objectives: masked language modeling and next sentence prediction. Longformer [5] is a variation of the Transformer architecture designed to handle long-range dependencies in text sequences more efficiently. Longformer addresses the limitations of standard Transformers, which have challenges with processing very long sequences due to computational constraints and memory limitations. Longformer achieves this by introducing a sparse attention mechanism that allows the model to focus on relevant tokens while ignoring irrelevant ones, thus reducing computational complexity. It processes long sequences of text efficiently by introducing local and global attention patterns. Instead of a fully connected attention matrix (quadratic computational complexity w.r.t sequence length), it has a sparse attention matrix that makes the computational complexity linear. In addition to BooksCorpus and English Wikipedia, it was also pretrained on CodeSearchNet, a dataset of 2 million (comment, code) pairs from open-source libraries hosted on GitHub [25]. In this thesis, I chose longformer-base-4096 [5] which is a BERT-like model started from the RoBERTa checkpoint with the Masked language modeling objective which supports sequences of lengths up to 4096 tokens.

3.4 PERPLEXITY

Perplexity is one of the common metrics for evaluating language models which computes how well a probability model predicts a sample and captures the degree of "uncertainty" or "randomness" of that model. A well-written document has a low perplexity, whereas a document containing only random word sequences has a high perplexity. The formula for perplexity (PP) is calculated as the inverse probability of the test set, normalized by the number of words:

$$PP(W) = \sqrt[N]{\frac{1}{P(w_1, w_2, ..., w_N)}}$$

Where N is the total number of words in the test set. $w_1, w_2, ..., w_N$ are the individual words and $P(w_1, w_2, ..., w_N)$ is the probability assigned to the entire test set by the language model.

3.5 OUT-OF-DISTRIBUTION DATA

In natural language processing, "out-of-distribution" (OOD) data refers to data samples that are significantly different from the distribution of data the model was trained on. This means that the data samples in the OOD set have different characteristics, patterns, or contexts compared to the data the model

has seen during training. OOD data exhibits topics or language usage that are not present or underrepresented in the training data. This could include domain-specific vocabulary, rare words, or unconventional grammatical structures. For instance, medical articles containing many special terms may be considered OOD data if a model is trained on sports articles. Similarly, in sentiment analysis, a model trained on product reviews might find it difficult to appropriately classify sentiment in tweets containing slang or informal language, which could potentially be considered OOD data. Various techniques, such as uncertainty estimation, anomaly detection, or adversarial testing, can be employed to identify OOD samples during model evaluation or deployment. Furthermore, fertility can be effective as it measures the diversity of subtokens produced by the tokenizer. A high fertility value may indicate that the given text has a high number of words that are not present in the vocabulary of the tokenizer.

3.6 TERM FREQUENCY–INVERSE DOCUMENT FREQUENCY

TF-IDF, or Term Frequency-Inverse Document Frequency, is a widely used method in information retrieval and natural language processing. It is a statistical technique that measures the relevance of a word in a document, with respect to a larger corpus of documents. The relevance score of a word is calculated by multiplying its frequency (number of times it appears in a document) by its inverse document frequency (logarithm of the number of documents in the corpus divided by the number of documents containing the word). High TF-IDF scores indicate words that are unique and informative to a document. This method has numerous applications, including search engine ranking, text classification, and document clustering.

$$TF - IDF(t,d) = TF(t,d) \cdot IDF(t)$$

$$TF(t,d) = \frac{f_{t,d}}{\sum_{k=1}^{n} f_{k,d}}$$

$$IDF(t) = \log \frac{N}{n_t}$$

Here, $f_{t,d}$ is the frequency of word "t" in document "d", n is the total number of words in document "d", N is the total number of documents in the corpus "C", and n_t is the number of documents in the corpus that contain the word "t".

3.7 NEAREST NEIGHBOR AND RECURSIVE NEAREST NEIGHBOR GRAPH

The nearest neighbor (KNN) [19] graph is a graph where each data point is connected to its k-nearest neighbors in a high-dimensional space. The high-dimensional space is typically represented using vector embeddings. Vector

embeddings and nearest-neighbor graphs are often used together in machine learning and data mining tasks to find similar items or data points. Vector embeddings can be used to create a nearest neighbor graph by placing them as nodes and connecting them to their k closest neighbors using a similarity metric such as cosine similarity, which calculates the cosine of the angle between two vectors to determine which direction the vectors are pointing in. Recursive KNN graphs, which is a novel method for this thesis, are created by using the KNN method recursively and finding the nearest neighbors of the neighbors of the original node and continuing the same process. A parameter that we call *nHops* in this project defines the number of times that this process is repeated (including the first KNN). Figure 3 illustrates an example of a recursive KNN graph's environment.

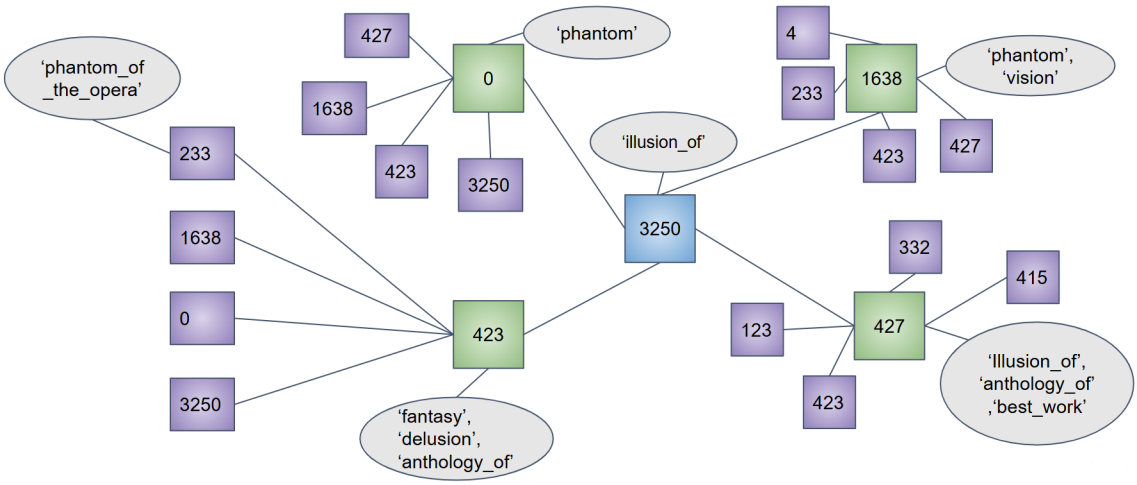


Figure 3: *Example of a node's environment in a Recursive KNN graph with K=4 and nHops=2 Each node is a document with the number as an identifier while some recursive KNN graph's nodes may occur multiple times in an environment. Each node has a set of keyphrases. it is expected a decent model to contain a few unique nodes in the neighborhood of each node and a high number of common keyphrases.*

The algorithm for creating a Recursive KNN consists of two parts which are creating a KNN (Algorithm 1) and recursively creating a recursive KNN (Algorithm 2) using the first graph.

Algorithm 1 Building KNN

Input: a matrix M storing the similarities between vector embeddings, nHops, K
Output: a dictionary D storing the closest neighbors

```
D ← ∅
for node ∈ M.rows do
    D ← D ∪ {node}
    L ← sorted row M[node] decreasingly w.r.t similarity scores of the node and
        nodes excluding the node itself
    D[node] ← L[0 : K]
    Neighbors ← D[node]
    counter ← 1
    D ← Building Recursive KNN(M, D, node, Neighbors, nHops, K, ,counter+1)
end for
```

Algorithm 2 Building Recursive KNN

Input: M, D, node, Neighbors, nHops, K, counter
Output: D

```
if counter ≤ nHops then
    for n ∈ Neighbors do
        D[node][counter][n] ← ∅
        L ← sorted row M[n] decreasingly w.r.t similarity scores of the node and
            nodes excluding the node itself
        D[node][counter][n] ← L[0 : K]
        Neighbors ← D[node][counter][n]
        D ← Building Recursive KNN(M, D, node, Neighbors, nHops, K,
,counter+1)
    end for
end if
```

3.8 KEY-PHRASE EXTRACTION

Keyphrase extraction is a natural language processing technique that involves automatically identifying and extracting the most important and relevant phrases or terms from a given text. The extracted key phrases capture the essential topics, concepts, and ideas discussed in the text, facilitating summarization, information retrieval, categorization, and organization of textual content. This process aids in condensing the main content of a text into a concise representation of its core meaning, without requiring readers to go through the entire text. There are several approaches to keyphrase extraction, ranging from traditional statistical methods to more advanced machine-learning techniques. Some common methods include:

- **Frequency-based Methods:** These techniques identify keyphrases by counting how often certain words or phrases appear in the text. Terms that appear frequently are often assumed to be important. However, these methods might not consider the context or semantic meaning of the terms.

- **TF-IDF:** As already stated, TF-IDF is a statistical measure that evaluates the importance of a term within a document relative to its importance in a collection of documents. Terms that are frequent in the current document but rare in the corpus receive higher TF-IDF scores.

- **Graph-based Methods:** These methods build graphs where nodes represent words or phrases, and edges represent relationships between them. Algorithms like TextRank [30], inspired by PageRank [33] for web page ranking, use graph analysis to identify keyphrases based on their centrality in the graph.

- **Topic Modeling:** Techniques like Latent Dirichlet Allocation (LDA) [8] can identify latent topics in a document collection and associate keyphrases with these topics. This method is especially useful for capturing the underlying themes within a set of documents.

The choice of method depends on the specific use case, available data, and desired level of accuracy. The method I used in the thesis is Textrank. It is an unsupervised keyword extraction algorithm inspired by PageRank, the algorithm used by Google to rank web pages. Textrank operates on the assumption that important words or phrases in a document are frequently mentioned and are often surrounded by other important words or phrases. The algorithm analyzes the graph structure of a text, with nodes representing words or phrases and edges representing the relationships between them. Here are the steps of this algorithm:

- **Preprocessing:** The input text is preprocessed to remove stopwords (commonly occurring words like "the," "is," "and," etc.) and punctuation marks. The remaining words or phrases are tokenized, and a graph representation of the text is constructed.

- **Graph Construction:** Each word or phrase in the text is represented as a node in the graph and the edges are added between nodes based on their co-occurrence within a predefined window of words. If two words appear within this window, a weighted edge is added between them. The weight of the edge is typically determined by some measure of semantic similarity, such as the frequency of co-occurrence.

- **Graph-based Ranking:** The TextRank algorithm uses an iterative ranking procedure similar to Google's PageRank algorithm after the graph has been

created. First, each node (word or phrase) is assigned a uniform importance score. The importance score of each node is updated iteratively based on the importance scores of its neighboring nodes. Nodes with higher importance scores will contribute more to the scores of their neighbors. The iterative process continues until convergence, where the importance scores stabilize.

- **Keyword Extraction:** The nodes (words or phrases) with the highest scores are chosen as keywords or key phrases once the graph-based ranking procedure has converged. The number of keywords extracted can be predefined or determined based on a threshold value.

TextRank is completely unsupervised since it doesn't rely on outside resources like dictionaries or labeled training data. It is robust to changes in text length and structure and can handle documents in any language. TextRank can handle enormous amounts of text because it is scalable and computationally effective.

4

METHODOLOGY

4.1 FLOW DIAGRAM

Figure 4 illustrates the comprehensive steps undertaken in our research. During inference time, only the right side of the chart, indicated by the blue direction, is needed. This means by having the corpus and its embeddings, the steps of extracting keyphrases, creating recursive KNN graphs, and conducting keyphrase analysis are done.

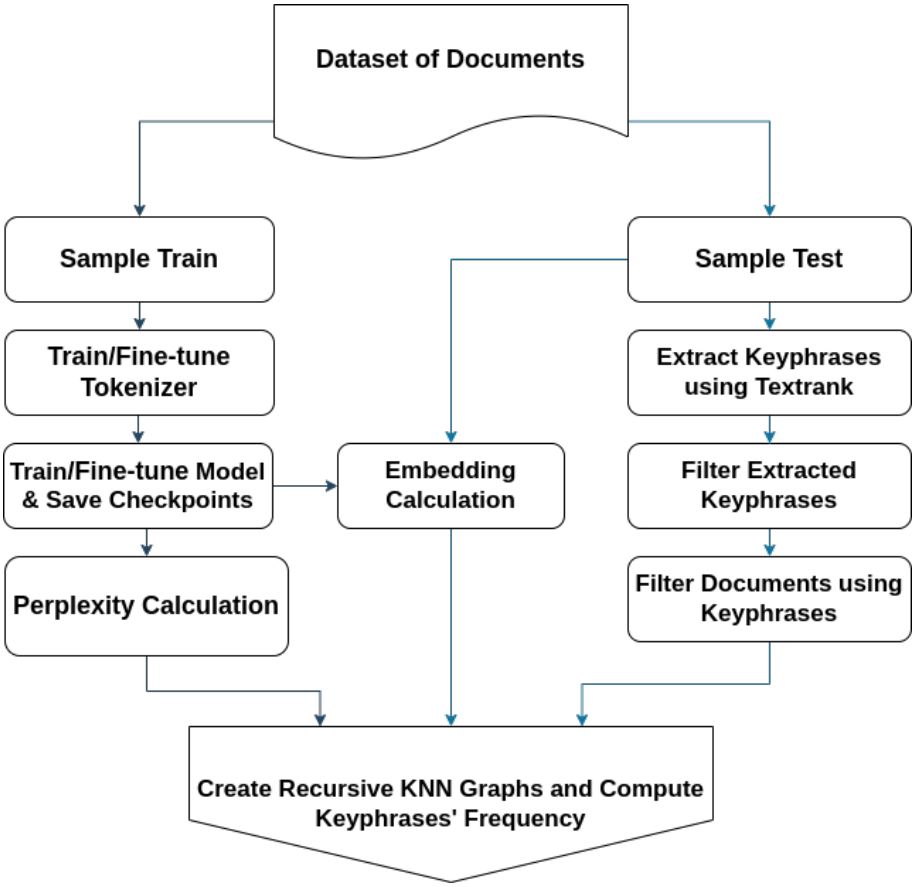


Figure 4: *Keyphrase Analysis Workflow*

4.2 DATA PREPROCESSING

The AmazonCats-13K dataset [7] has been selected for training and fine-tuning the tokenizer. This dataset includes a training set and a test set with various features, which will be described in section 5. In this thesis, the 'content' column is used, which contains product descriptions from Amazon. The preprocessing and subset selection methods may vary depending on the stage of the algorithm.

For additional experiments, various datasets are chosen. These datasets consist of xnli [14], a dataset for natural language inference, x_stance [49] and MedQuad [6], two datasets for question-answering, FEVER [46], a dataset consisting of claims created by altering sentences extracted from Wikipedia, and emotional-reactions-Reddit [43] which contains emotional reactions from Reddit. The contents of the question-answering datasets are formed by combining the questions and answers, and the emotional reactions contents include the concatenation of seeker and response posts. For the xnli dataset, the hypothesis and premise sentences are concatenated before proceeding to the subsequent steps. In the x_stance datasets, as only the text file was provided, certain common terms such as 'Thema' and 'Meine Meinung ist (dafür oder dagegen)' are excluded from the text file. These terms are unrelated to the dataset and could potentially affect the keyphrase analysis.

4.2.1 *Tokenizer*

To train or fine-tune the tokenizer, the training set is utilized after removing instances with empty contents (NaN values in the 'content' column). The remaining instances' contents are then employed without any preprocessing.

4.2.2 *Embeddings Calculation*

In order to generate the recursive KNN graphs for keyphrase analysis, embeddings are required. To calculate the embeddings for the AmazonCats dataset, the subsets of 10,000 instances are chosen from the test set. Similarly, instances with no content are removed. The contents generated for the other datasets are saved in text files and then provided to the chosen language model to obtain their embeddings.

4.2.3 *Keyphrase Extraction and Recursive KNN Graph Creation*

For extracting the keyphrases as well as creating the recursive KNN graphs, the same subset of the AmazonCats test set as well as the contents generated for the evaluation datasets are given to the keyphrase extraction algorithm to derive

the keyphrases of each set. This is because the algorithm initially encodes the sets using a tokenizer.

4.3 TOKENIZER TRAINING/FINE-TUNING

In this thesis, the focus of the research is not on the impact of different tokenizers. Instead, the tokenizers are fine-tuned and trained to align with the models being used. As a result, the combination of the tokenizer and the model is treated as a complete black-box model.

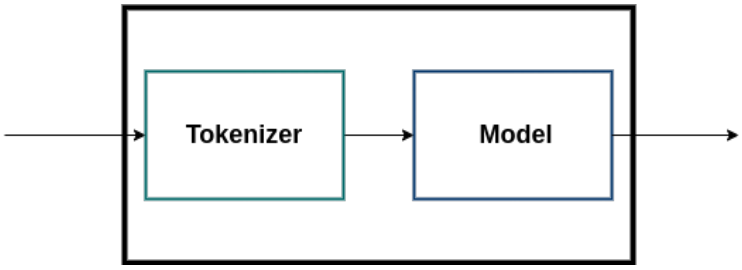


Figure 5: *The black-box model*
The model, together with the tokenizers, is regarded as a black-box model.

Firstly, a pretrained tokenizer-model pair is utilized for fine-tuning, while another pair is initialized randomly and trained on the AmazonCats dataset. The chosen tokenizer is longformer-base-4096 to match with the corresponding model. For fine-tuning, the tokenizer is loaded and further trained on the entire training set using HuggingFace's Transformers library [48] via the *train_new_from_iterator* function. The resultant outputs are stored in several files:

- **vocab.json:** Including the complete vocabulary of the tokenizer.
- **added_tokens.json:** TListing the additional tokens including "</s>", "<mask>", "<pad>", "<s>", and "<unk>".
- **special_tokens_map.json:** Providing mappings for special tokens:

```
"bos_token": "<s>",
"cls_token": "<s>",
"eos_token": "</s>",
"mask_token": "<mask>",
"pad_token": "<pad>",
"sep_token": "</s>",
"unk_token": "<unk>"
```

- **merges.txt:** Containing a series of merge operations for the tokenizer's subword tokenization.

- **tokenizer_config.json:** Incorporating the tokenizer's configuration details such as its class, maximum length, and decoder for added tokens.

- **tokenizer.json:** A comprehensive file encompassing the complete tokenizer information.

To train a tokenizer from scratch, the process begins by creating a BpeTrainer object from Huggingface's tokenizer library [47]. This trainer trains models employing the BPE (Byte-Pair Encoding) algorithm [42], a widely-used subword tokenization method utilized in models such as GPT [38] and RoBERTa [27]. BPE breaks down words into subword units based on their frequency within a given corpus. Through the iterative merging of the most frequently occurring pairs of characters or character sequences, the algorithm progresses until a predefined vocabulary size is reached. This process allows for better handling of out-of-vocabulary words and improves the overall performance of language models. The tokenizer construction involves giving the special tokens (as previously outlined), a vocabulary size of 52,000, along with a pre-tokenizer and decoder. For this purpose, the ByteLevel pre-tokenizer and decoder are selected due to their simple and straightforward approach, replacing all bytes in a given string with corresponding representations and segmenting them into words. Then, the tokenizer undergoes training on the complete training set.

However, when saving this tokenizer, only the tokenizer.json file is created. To ensure the creation of all necessary files essential for the transformer model to load the tokenizer, a PreTrainedTokenizerFast instance is generated from HuggingFace's transformers library. This instance is configured to load the trained tokenizer, special tokens, and the maximum length of the model. Consequently, saving this instance would result in the creation of all needed files for integration with the transformer model.

4.4 LANGUAGE MODEL TRAINING AND EMBEDDINGS CALCULATION

In order to compare various models, I initiated the process by selecting a transformer-based model and fine-tuning it using a training subset of the AmazonCats-13k dataset, comprising 150,000 product descriptions. For this task, I chose the longformer-base-4096 model, which is derived from the RoBERTa checkpoint and is similar to BERT architecture. The fine-tuning of the Longformer model involves employing masked language modeling, where the objective is to predict masked words or tokens within a given text. To execute the fine-tuning process, I utilized the *run_mlm.py* script available on HuggingFace's GitHub repository [29] with the fine-tuned tokenizer. The fine-tuning procedure involves training the model for 80 epochs and the checkpoints are saved every 3000 steps

(gradient updates) throughout the training process. The detailed setup utilized for training the models is documented in the *Experiments and Results* section.

Additionally, another Longformer model is initialized randomly and trained using the trained tokenizer with a configuration similar to the aforementioned fine-tuning process. Afterward, the final models and checkpoints at various training steps are saved. The evaluation set is then fed into the models' encoders to compute embeddings. The embedding vectors of the input tokens are derived from the hidden states of the last layer of the models, and they are averaged to obtain text embeddings, each having a dimensionality of 768.

4.5 KEYPHRASE EXTRACTION

Following the generation of embeddings, I focused on examining the evaluation datasets. To ensure data quality, the null values are removed from the test dataset. Subsequently, the goal shifts towards extracting keyphrases from these evaluation sets.

For this purpose, I employed the pke (Python Keyphrase Extraction) [9] module's TextRank algorithm, a prominent graph-based ranking model for phrase extraction within the field of NLP. The underlying principle of TextRank revolves around the construction of a word network. Notably, connections are established between words that appear in immediate succession. The strength of these connections is determined by the frequency with which the two words co-occur within close proximity in the text.

The TextRank algorithm requires certain parameters to be passed, including the language, part-of-speech tags, and the language model. The part-of-speech tags utilized are ADJ (adjective), NOUN, PROPN (proper noun), VERB, and ADP (adposition). The language is inferred based on the input corpus in the script, which could be either German or English in the context of this thesis. Accordingly, the *en_core_web_sm*, a small English pipeline trained on written web text such as blogs, news, and comments [3], or *de_core_news_sm*, a small German pipeline trained on the news [2], spaCy language model is selected to process the corpus. Given the large size of the datasets and the limitations in the processing capacity of the spaCy language model, the documents are processed in batches of 6000. The extraction process yields keyphrases along with their corresponding importance scores, which are then saved in dataframes with two columns of "Phrase" and "Score". These dataframes are subsequently merged to result in unique keyphrases along with their respective scores.

4.6 KEYPHRASE FILTERING

After the initial extraction of keyphrases, a series of filtering operations are conducted to retain only the most important keyphrases:

1. **Phrase Length Restriction:** Keyphrases are limited to a maximum length of 4 words. Any keyphrase exceeding this limit is discarded.

2. **TF-IDF Thresholding:** Through TF-IDF analysis, the weights of words within the extracted phrases are computed. A threshold criterion is then applied, set to the 80th percentile of the TF-IDF values distribution. Keyphrases containing at least one word with a TF-IDF value surpassing this threshold are retained, while others are removed.

3. **Phrase Frequency-based Pruning:** This step involves preserving only the top decile (i.e., the highest 10%) of phrases based on their frequency. Frequent phrases are determined using methods such as FP-Growth and are retained for further steps.

4. **Word Frequency-based Pruning:** In the final filtering stage, only the top percentile (i.e., the uppermost 1%) of phrases are preserved, based on word frequency. The frequencies of words within the retained phrases are calculated across the evaluation set, and keyphrases containing at least one frequently occurring word are retained.

4.7 DOCUMENT FILTERING

Following the initial step of filtering out keyphrases, the evaluation dataset undergoes further processing and organization. Specifically, only documents containing at least one keyphrase from the set of filtered keyphrases are retained. Thus, a dataframe is constructed with two columns: "text" and "key_phrases", representing the remaining contents and their corresponding keyphrases, respectively. This dataframe facilitates the analysis of the relevant textual data alongside their associated keyphrases for the following steps.

4.8 RECURSIVE KNN GRAPH CREATION

After extracting and filtering keyphrases alongside documents, the recursive KNN graphs are constructed. This involves reading the calculated embeddings of the evaluation set and specifying values for K (number of nearest neighbors) and nHops (number of Hops for recursion).

Initially, embedding vectors of the remaining documents are selected as nodes for the graph. Using the scikit-learn library [35], cosine similarities between all pairs of these vectors are computed. Then, for each node, its K closest neighboring nodes from other documents are identified, forming the first layer of its recursive KNN environment where nHop is equal to 1. This process is repeated for each neighbor, generating the following layers. The creation of each recursive KNN environment continues until the predetermined nHop is reached. During each nHop, various features such as the number of neighbors, pairwise cosine similarities, and keyphrases within the environments are derived and updated. Once the graph is created, properties of its environments, including the mean cosine similarity and keyphrase frequencies, are computed from the stored features. This results in the gaining the environment-based properties of the graph

These properties provide insights into the distribution of various features within the generated recursive KNN graphs, enabling comparisons between different graphs' environments. The plots of these properties for different models and datasets are shown in the *Results* section. Moreover, the 2% environments with the most and least nodes are selected and the respective diagrams are plotted to check the patterns and behaviors of the largest and smallest environments.

4.9 KEYPHRASE ANALYSIS

To understand the overall distribution of keyphrases across the entire graph, the frequencies of keyphrases within all environments are aggregated to form a list of unique keyphrases along with their respective frequencies. This list is sorted based on the frequencies and then visualized.

For the embeddings derived from different checkpoints of the longformer models on the AmazonCats dataset, the differences in distributions between checkpoints are computed and plotted for further analysis and deeper insights. Specifically, each checkpoint's distribution is subtracted by that of the preceding checkpoint, as well as the distribution obtained from the pretrained model (without any fine-tuning). To ensure compatibility for subtraction, padding with zeros is applied to the smaller distribution list. The findings and the plots are shown and discussed in the *Result* section.

4.10 OUT-OF-DISTRIBUTION SELECTION

The effectiveness of the keyphrase analysis method is also evaluated across datasets from different domains. The method's ability to recognize domain shifts in these datasets and its correlation with model performance is investigated.

These datasets significantly differ from the AmazonCats dataset used for model fine-tuning and training, indicating that they are out-of-distribution for the models.

After collecting datasets from various domains such as medical, fine-tuned and trained tokenizers are utilized to identify those distinct from the AmazonCats dataset. In addition to analyzing the number of words and distribution of document lengths in each dataset, the tokenizers' performances are also assessed. This involves processing each dataset using the tokenizers, followed by calculating the tokenizers' fertilities. Therefore, the total number of generated tokens (including duplicates) is divided by the total number of words obtained by splitting the text using spaces. Selecting the datasets with significantly different fertilities results in the selection of the datasets of MedQuad [6], FEVER [46], and emotional-reactions-Reddit [43].

5

DATA

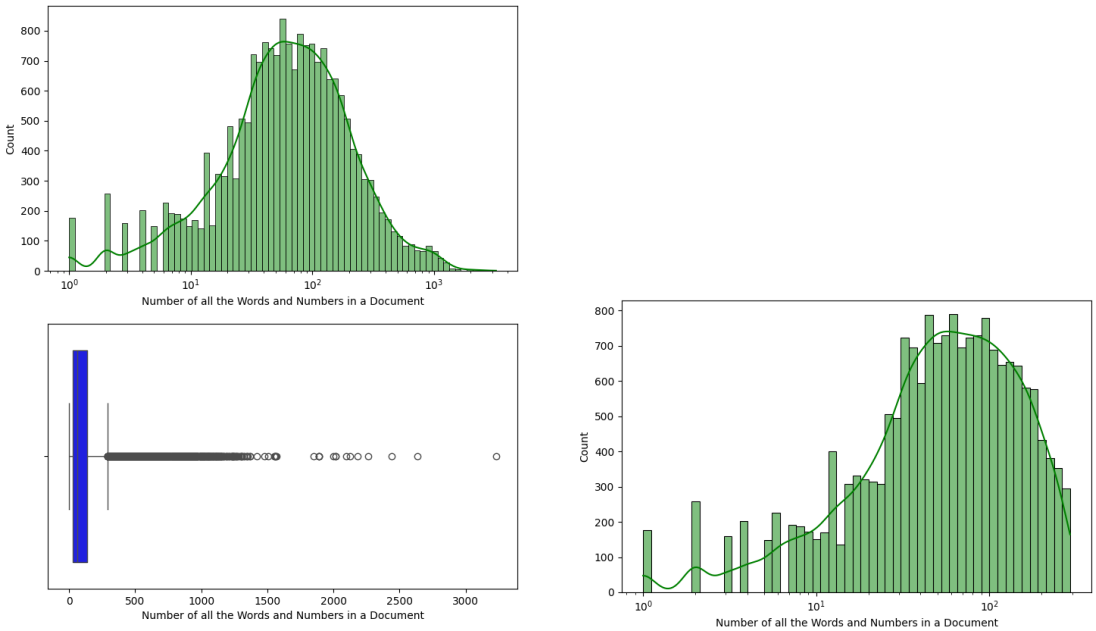
5.1 AMAZONCATS-13K

The dataset we used is called Amazon Cat-13k [7]. It is a benchmark for extreme multi-label text classification (XMTC) tasks with 13,330 Amazon product categories and 1,186,239 (English) product descriptions.

The AmazonCat-13k dataset is a large and complex dataset where the properties are not evenly distributed, some properties are more common than others, it has an unbalanced label distribution, and there are errors and inconsistencies in the data. Nevertheless, it is a valuable resource for machine learning research as it accurately mimics real-world XMTC settings where data points contain numerous appropriate labels. Yet, the label space is large and frequently sparsely populated, with an average of 5.44 labels per product. For this project, I randomly selected a subset of this dataset with 150,000 documents from the training set and used it for training, whereas three distinct subsets of 10,000 documents were selected from a sample of the test set consisting of 100,000 instances and used for evaluation. Plots 6a and 6b show the distributions of document lengths which are the counts of words and numbers before and after outlier removal using the Interquartile range (IQR). A document is considered an outlier if its value is more than 1.5 times IQR above the third quartile or below the first quartile.

5.2 MEDQUAD

One out-of-distribution selected dataset is MedQuad [6], also known as the Medical Question Answering Dataset. It is a valuable resource for training and evaluating NLP models in the medical domain. It contains a collection of around 47,457 question-answer pairs specifically focused on medical topics. These questions and answers are curated from 12 websites of the National Institutes of Health (NIH) in the US, including popular sites like cancer.gov and MedlinePlus, and they cover 37 question types including Treatment and Diagnosis.



(a) *Distributions of document lengths before removing outliers* ***(b)*** *Distributions of document lengths after removing outliers*

Figure 6: *distribution of document length*

The scope of the MedQuad dataset goes beyond just the questions and answers themselves. It also incorporates rich annotations for each question, providing additional information that can be useful for NLP tasks. These annotations include the type of question being asked, the specific medical entity the question focuses on (Disease, Drug, Other), synonyms for the question itself, and unique identifiers and semantic types from the Unified Medical Language System (UMLS). This test collection can be used to evaluate the effectiveness of information retrieval (IR) and question-answering (QA) systems on medical-based questions. I selected a subset of 16,400 documents for my thesis, which are the questions and answers concatenated.

5.3 FEVER

Another out-of-distribution selected dataset is the FEVER [46] dataset (Fact Extraction and VERification) which tackles a distinct NLP challenge: verifying the accuracy of claims against textual evidence. FEVER offers a massive collection exceeding 185,445 claims. These claims are constructed by modifying sentences extracted from Wikipedia. The key aspect here is that human annotators then verify these claims against the corresponding Wikipedia introductory sections,

not the original sentence the claim originated from. This verification process assigns each claim one of three labels: "Supported" if the claim is factually correct based on evidence found in Wikipedia, "Refuted" if the claim contradicts evidence, and "NotEnoughInfo" if there's insufficient information in the provided Wikipedia passage to determine the claim's accuracy. Notably, for claims labeled "Supported" or "Refuted," the annotators also pinpoint the specific sentences within Wikipedia that serve as evidence for their judgment.

With its size and rich annotations, the FEVER dataset is valuable for training NLP models that can assess the credibility of claims and find supporting evidence within textual sources. I selected a subset of 20,000 documents in this thesis, in which the contents are the generated claims for each instance.

5.4 EMOTIONAL-REDDIT

The additional dataset I gathered for my thesis is the emotional-reactions-Reddit [43], which focuses on empathy expressed in text-based mental health support. It contains 3084 pairs of posts and responses from Reddit, where the posts are from individuals seeking support and the responses are from other users. These pairs have been annotated using the proposed empathy framework in the paper, indicating the level of empathy in the responses relative to the seeker's posts, along with supporting evidence for each level. This dataset is valuable for studying how empathy is conveyed in different online platforms and consists of everyday language that is not overly complex. To create the dataset, the seekers' posts and responses are concatenated to form the content.

5.5 XNLI AND X_STANCE

The two additional datasets were chosen to showcase the evaluation method's performance with different language models and languages. The xnli [14] dataset focuses on Natural Language Inference (NLI), which involves predicting if a sentence implies, contradicts, or has no relationship with another sentence. For this work, a subset of 20,000 German premise and hypothesis pairs were selected and combined for analysis.

The x_stance [49] dataset is utilized to train and evaluate stance detection systems. It contains political questions and written comments from candidates expressing their opinions on these questions. The comments are in German, French, and Italian, while the questions are available in all three languages as well as English. In this work, a subset of 20,000 German question-and-answer pairs were chosen and concatenated.

5.6 GENERAL PROPERTIES OF THE DATASETS

Table 1 presents the key characteristics of the datasets, including the distributions of document lengths before and after removing outliers as well as the counts of all the words and unique words. It is important to note that the outlier documents are not removed for the graph creation and keyphrase analysis. These document lengths include the counts of words and numbers, as well as the counts of words and unique words.

Table 1: *General properties of the datasets*

Datasets	Document Length						Number of Words	
	Before			After			All	Unique
	Mean	Min	Max	Mean	Min	Max		
AmazonCats-13k	115	1	3230	77	292	1	2,083,492	906,207
MedQuad	107	4	977	95	258	4	3,438,383	74,510
FEVER	8	2	53	7	13	2	161,823	40,873
Emotional-Reddit	58	10	636	52	10	126	251,161	19,878
xnli	26	3	140	25	3	54	730,001	43,544
x_stance	37	11	100	36	72	11	854,342	61,017

6

EXPERIMENTS

6.1 OUT-OF-DISTRIBUTION DETECTION

The primary dataset used for training and fine-tuning the models is AmazonCats-13k. Additional datasets from various domains, different from AmazonCats-13k, were selected to assess the effectiveness and performance of the keyphrase analysis method. Each dataset was processed using three different tokenizers to evaluate the fertilities of the tokenizers. These tokenizers include the longformer-base-4096 pretrained tokenizer, a fine-tuned tokenizer, and a tokenizer trained from scratch. The fertility of each tokenizer was calculated by dividing the number of encoded subtokens by the total number of words obtained by splitting the text using spaces. To accomplish this, five random subsets with each 1500 instances are chosen from a dataset. The number of words and subtokens in each subset is derived, and the fertility is calculated. The final results are obtained by averaging these calculations. The results are summarized in Table 2. It should be noted that the xnli and x_stance datasets are in German language. These datasets are used to evaluate the keyphrase analysis method on different models and languages.

Table 2: *The obtained fertility values for datasets from various domains*

Datasets	Fertility		
	Pretrained	Fine-Tuned	Scratch
AmazonCats-13k	3.1	3	2.9
MedQuad	3.22	3.39	3.28
FEVER	1.3	1.29	1.29
Emotional-Reddit	1.51	1.5	1.49
Turkish	2.55	2.56	2.54
Arabic	6.81	12.01	1.18

The selected datasets exhibit varying fertility results, indicating the presence of different words and tokens in each dataset. Two datasets, comprising Turkish and

Arabic words, were employed to verify the plausibility of fertility calculations. Both datasets exhibited high fertility values as the phrases had not been previously encountered by the tokenizer. In the case of the Arabic dataset, the small fertility value was primarily due to a high prevalence of unknown words, thus encoding them to the unknown token, in the model trained from scratch. Due to the more common and simpler language used in the FEVER and Emotional-Reddit datasets, their fertility results are lower compared to the AmazonCat-13k and Medquad datasets. This is because the latter datasets contain more complex and specialized words.

6.2 MODEL TRAINING AND FINE-TUNING

The models are trained and fine-tuned for 80 epochs with a learning rate of 1e-4, a MLM probability of 0.15, and a batch size of 80. Additionally, there were other configurations used for the fine-tuning process in the *run_mlm.py* script:

```
--gradient_accumulation_steps 20
--line_by_line True
--max_seq_length 512
--mlm_probability 0.15
--model_name_or_path allenai/longformer-base-4096
--model_type longformer
--pad_to_max_length True
--per_device_eval_batch_size 80
--per_device_train_batch_size 80
--preprocessing_num_workers 100
--save_steps 3000
--save_strategy steps
--save_total_limit 10
--warmup_steps 500
```

To train the model, a new model is initialized for training because only the *model_type* is defined and there is no specific *model_name_or_path* provided. The resulting checkpoints are at steps 3000, 6000, 9000, and 12000. The resulting models have 12 hidden layers and attention heads with an attention window size of 512. The models also have a hidden dropout probability of 0.1, a hidden size of 768, maximum position embeddings of 4096, and a vocabulary size of 52000.

6.3 KEYPHRASE EXTRACTION AND FILTERING

The subsets of the datasets that have been selected are processed using the keyword extraction method. This method helps identify the appropriate SpaCy

language model for either German or English languages. The keywords are then extracted and saved. The documents are grouped into batches, with each batch containing a maximum of 6000 documents. The documents of each batch are joined into a single string, along with the part-of-speech tags (ADJ, NOUN, PROPN, VERB, ADP), and the language are then passed to the pke's TextRank algorithm with the window size of 2 for connecting two words in the graph. For each batch, a maximum of 15,000 keyphrases, along with their importance scores, are extracted and saved in a dataframe. Finally, the dataframes from each batch are averaged, resulting in a final dataframe that contains unique keyphrases and their scores.

The extracted keyphrases undergo several filtering steps to retain the most important ones. These filtering steps include limiting the keyphrases to a maximum length of 4 words, applying TF-IDF thresholding at the 80th percentile, preserving only the top 10% most frequent phrases, and retaining phrases with the highest 1% frequency of words. The TF-IDF values are obtained using Sklearn's *TfidfVectorizer* with *min_df* (document frequency) equal to 3. The numbers of keywords before and after filtering for the datasets are presented in Table 3. The documents in each dataset are filtered to include only those that contain the filtered keyphrases.

Table 3: *The number of keyphrases before and after filtering*

Datasets	Keyphrases		Docuemnts	
	Before	After	Before	After
	224,501	12,767	10,000	6795
AmazonCats-13k	224,501	13,145	10,000	6746
	224,501	12,013	10,000	6709
MedQuad	41,709	1894	16,412	14,646
FEVER	100,042	2110	20,000	18,186
Emotional-Reddit	15000	647	3084	3080

6.4 CONFIGURATIONS FOR KEYPHRASE ANALYSIS

The experiments consist of two main steps. Firstly, the creation of recursive KNN graphs using the AmazonCats embeddings obtained from various checkpoints for plotting the distributions of the frequencies the keyphrases. Secondly, the encoding of texts from other datasets using the most recent checkpoint to generate recursive KNN graphs and plot the distributions of keyphrases' frequencies.

6.4.1 *AmazonCats-13k*

After training and fine-tuning the Longformer model using the AmazonCats training dataset and saving the checkpoints, these checkpoints and the pretrained model are utilized to compute the embeddings of the test dataset. Firstly, a sample of 100,000 documents from the test set is chosen, and the keyphrases are extracted from these documents. Next, three distinct subsets of 10,000 documents each are selected from this sample. For each one, the documents are filtered based on the filtered keyphrases, and the embeddings of the remaining documents are employed to construct recursive KNN graphs using the cosine similarities between the embeddings. To examine the impact of K and nHops, the experiments are conducted with the following configurations:

- K=3 & nHops=3
- K=3 & nHops=4
- K=4 & nHops=4
- K=5 & nHops=5

The constructed graphs have different environments with varying numbers of unique nodes due to the possible repetition of neighbors across different nHops and layers. After creating each graph, the frequency of each keyphrase is aggregated across all graph environments, resulting in a set of unique keyphrases and their frequencies. This set is then used to generate different plots:

- Distribution of the frequencies of all keyphrases
- Differences in the distribution of the frequencies of the keyphrases between the embeddings from different checkpoints and pretrained models.

The plots are presented in the *Results* section and are examined to identify any notable patterns and behaviors. This analysis includes investigating both the pretrained model and the checkpoints at different stages of training. In order to establish a correlation between these plots and the performance of the models, perplexities are calculated for both the test and train sets using these models

6.4.2 *Additional Datasets*

After selecting the datasets MedQuad, FEVER, and Emotional-Reddit from different domains, the embeddings of these datasets are generated using the most recent checkpoint at step 12000. These embeddings are then utilized to

construct the recursive KNN graphs. For each dataset, the dataframe containing its keyphrases is combined with the created graph to calculate the set of unique keyphrases and their frequencies. However, in this case, only the distribution of the frequencies of all keyphrases is plotted. Consequently, the distributions of the different datasets, along with the AmazonCats dataset where the embeddings are derived from the same checkpoint, are plotted together to analyze the effect of domain shift.

Regarding the xnli and x_stance datasets, they are analyzed separately as they are in German and generated using different models. The embeddings of each dataset are obtained using two different models, and the performances of these models on different downstream tasks are known.

For the xnli dataset, it was evaluated with the NLI objective using the following models:

- model 1: BLOOM-CLP German [32] with an accuracy value of 0.51.
- model 2: German GPT2-XL model [28] with an accuracy value of 0.488.

And for the x_stance dataset, it was evaluated with the stance detection objective using the following models:

- model 1: German GPT2-XL model [28] with f1 value of 0.34.
- model 2: Multilingual GPT model [44] with f1 value of 0.49.

For each dataset, the dataframe that contains the keyphrases is first loaded. Then, a recursive KNN graph is created and the set of unique keyphrases along with their frequencies is calculated. Moreover, the f1 scores of the corresponding two models for each dataset are provided. To analyze the relationship between keyphrase analysis and performance, the differences in the distribution of keyphrase frequencies between the embeddings from the two corresponding models are plotted alongside the distribution of the frequencies of all keyphrases. These patterns are also compared to the ones found for the AmazonCats dataset, using different checkpoints, to analyze how changes in models and languages affect the results.

7

RESULTS

This section presents the outcomes of various experiments, which include evaluating different models on the same dataset as well as applying the same model to different datasets.

7.1 DIFFERENT MODEL VARIANTS ON THE SAME DATASET

Initially, this section illustrates and discusses the performance of keyphrase analysis using the same dataset across various models.

7.1.1 *AmazonCats*

The AmazonCats dataset was employed to fine-tune and train a Longformer model, with checkpoints saved every 3000 steps. Figure 7 illustrates the accuracy and loss metrics of the checkpoints on the evaluation dataset.

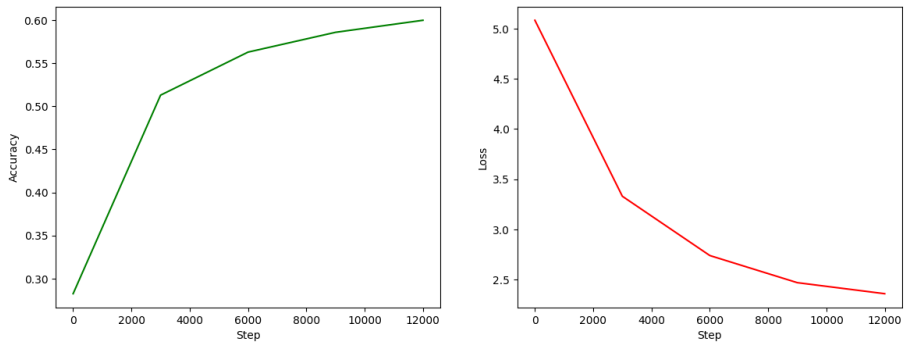


Figure 7: *Checkpoints' scores on the evaluation set*

Using these checkpoints, embeddings for three distinct subsets of 10,000 test documents were calculated and used to conduct keyphrase analysis. For each subset, keyphrases are sorted in non-decreasing order by their frequencies and plotted. Consequently, different distributions may display different keyphrases on their x-axis, as they are independently sorted for each sample. Figures 8 displays the keyphrase distributions for these subsets under different configurations for

creating recursive KNN graphs, revealing that the general patterns are similar with only minor variations.

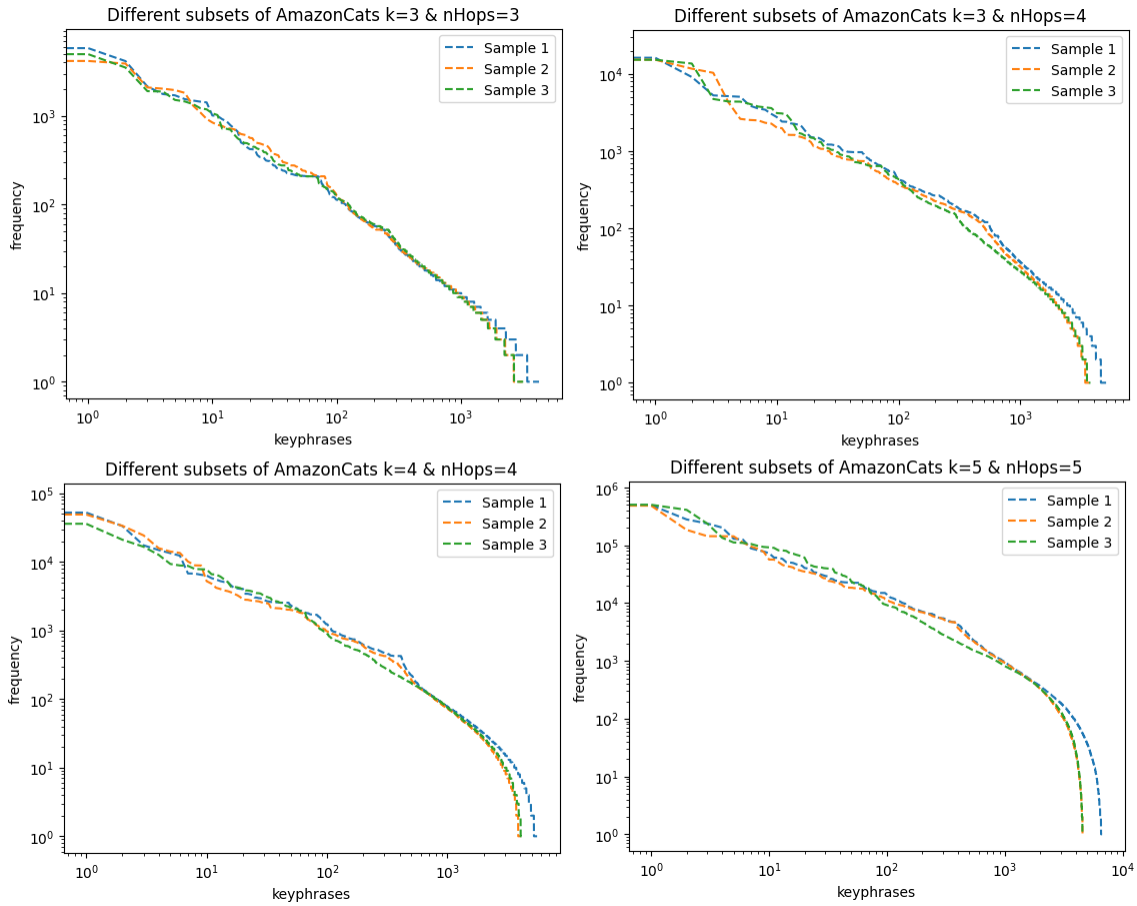


Figure 8: *Keyphrase distributions for different subsets of the AmazonCats dataset*
The x-axis represents independently sorted keyphrases, while the y-axis indicates the frequencies of the keyphrases.

For the subsequent experimental results, the average of these three distributions is calculated and plotted. Figures 9 displays the average distributions of the embeddings generated by the pretrained Longformer model and the checkpoints at steps 3000, 6000, 9000, and 12000. For the following plots, the x-axis represents independently sorted keyphrases and the y-axis includes the frequencies of these phrases. Figures 10 show the distributions of the 0.2 most and least frequent keyphrases for K=4 and nHops=4, where the distinct patterns for different checkpoints are more evident. As before, the numbers on the x-axis represent the indices of the sorted keyphrases, which are independent for each checkpoint.

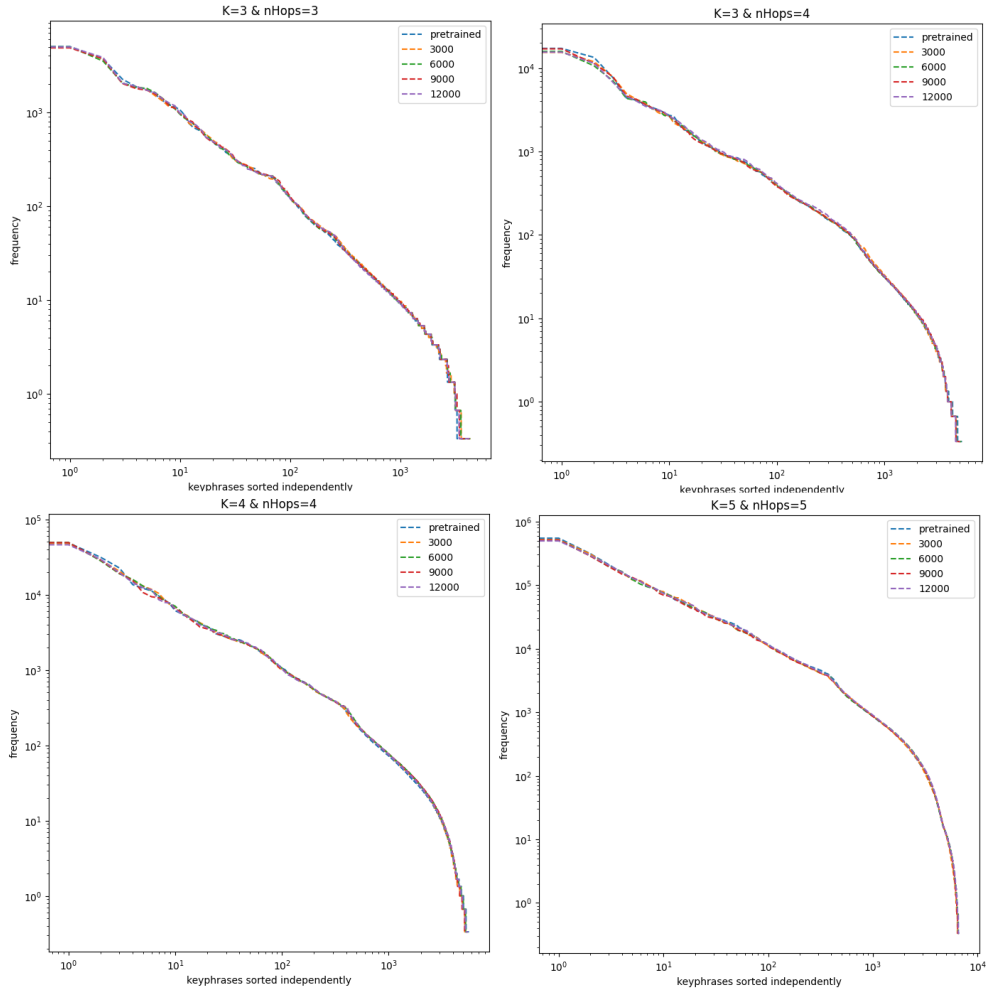


Figure 9: *Frequency distributions of keyphrases across different checkpoints*

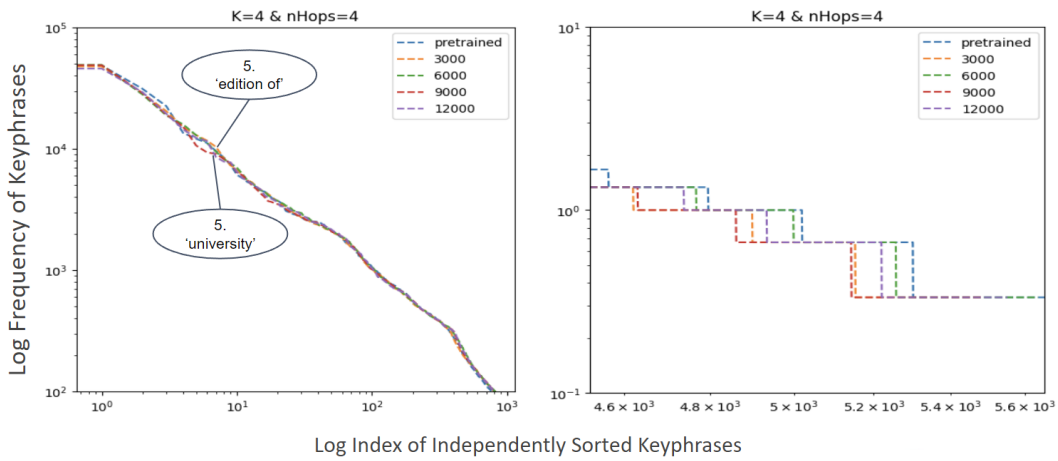


Figure 10: *distributions of 0.2 most and least frequent keyphrases across different checkpoints for K=4 & nHops=4*

To more clearly understand the distinctions between distributions, the differences between each checkpoint's embeddings and those of the preceding checkpoint, as well as the pretrained model, are calculated and illustrated. The resulting figures depict these differences, with red plots representing comparisons to the previous checkpoint and purple plots indicating comparisons to the pretrained model. Upon examining the sorted keyphrases for each model, it becomes apparent that the most frequently occurring keyphrases (the left sides of the figures 9) are typically general and not specialized, with low importance scores. Conversely, the keyphrases in the figures' middle and right sections represent the desired keyphrases. An example is provided below:

```
Most_Frequent: {
    "of the": 80431, "one of": 13439, "edition of": 6761, "of to": 6502, ...}
Frequent: {
    "intersection of highways": 2545, "manufacturer of": 1832,
    "illusion of": 775, "fateful night of april": 743,
    "anthology of": 582, "of australia": 458, ...}
Least_Frequent: {
    "international school of management": 67,
    "of fragrances": 55, "models made of vinyl": 34,
    "hint of traditional styling": 26,
    "individual nurturance of children": 10, "rhythm of life": 3, ...}
```

To better analyze the desired keyphrases, the initial 200-500 keyphrases were omitted and the figures were plotted again. Figure 11 displays plots with all keyphrases, while figures 14, 13, 12, and 15 show results for various configurations after cutting off the initial phrases. The presence of more positive values in many of these plots suggests that the frequencies of important keyphrases increase in later checkpoints. This indicates a focusing effect through fine-tuning of the models, where later checkpoints demonstrate improved accuracy in clustering documents when creating recursive KNN graphs. As documents with similar content are grouped together, the frequency of important keyphrases increases, and the frequency of common phrases decreases. The outcomes from the model trained from scratch are similar and exhibit the same pattern. Consequently, they are not included in the report to avoid redundancy.

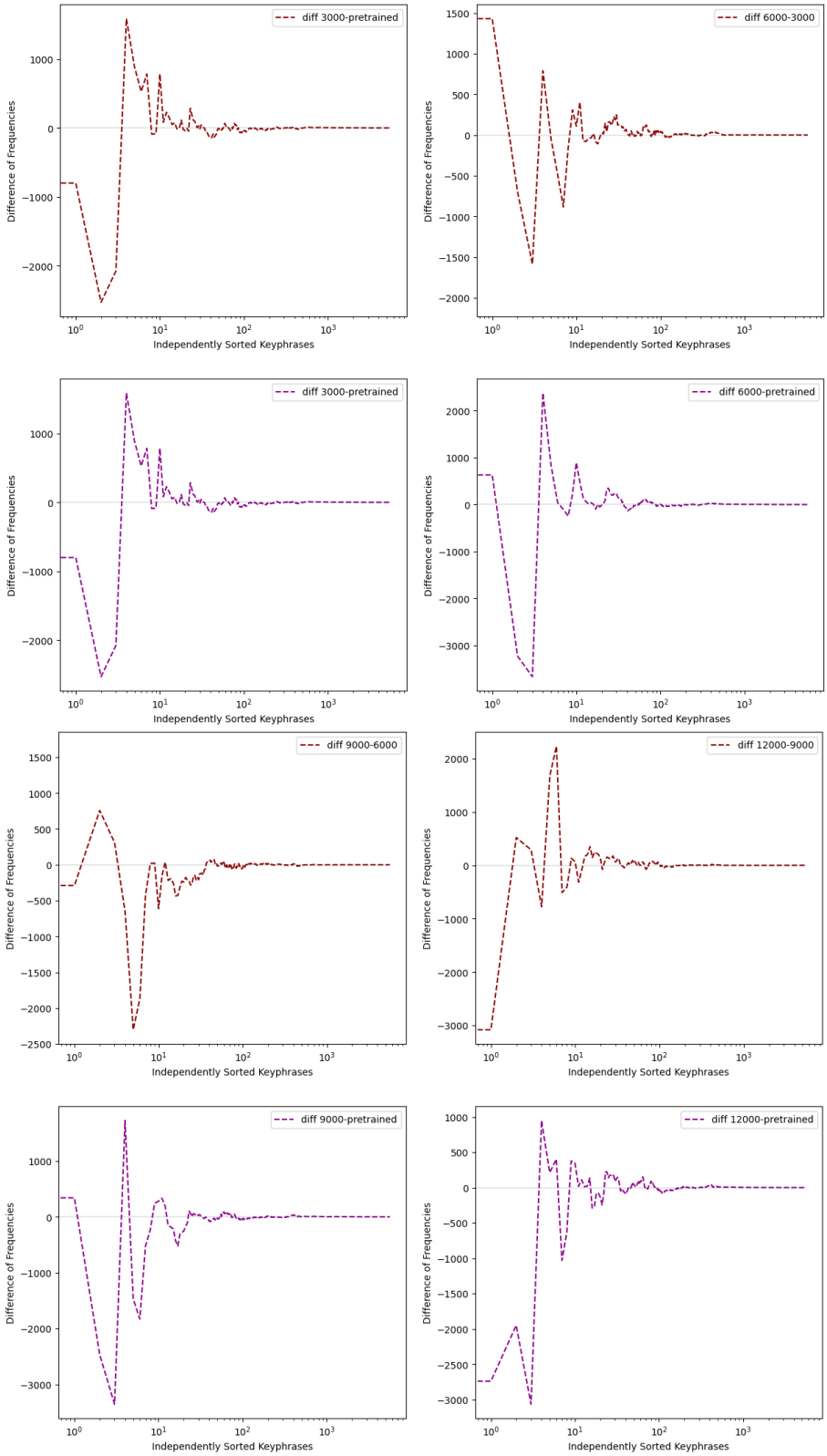


Figure 11: *Difference in the frequencies of all keyphrases with K=4 and nHops=4*

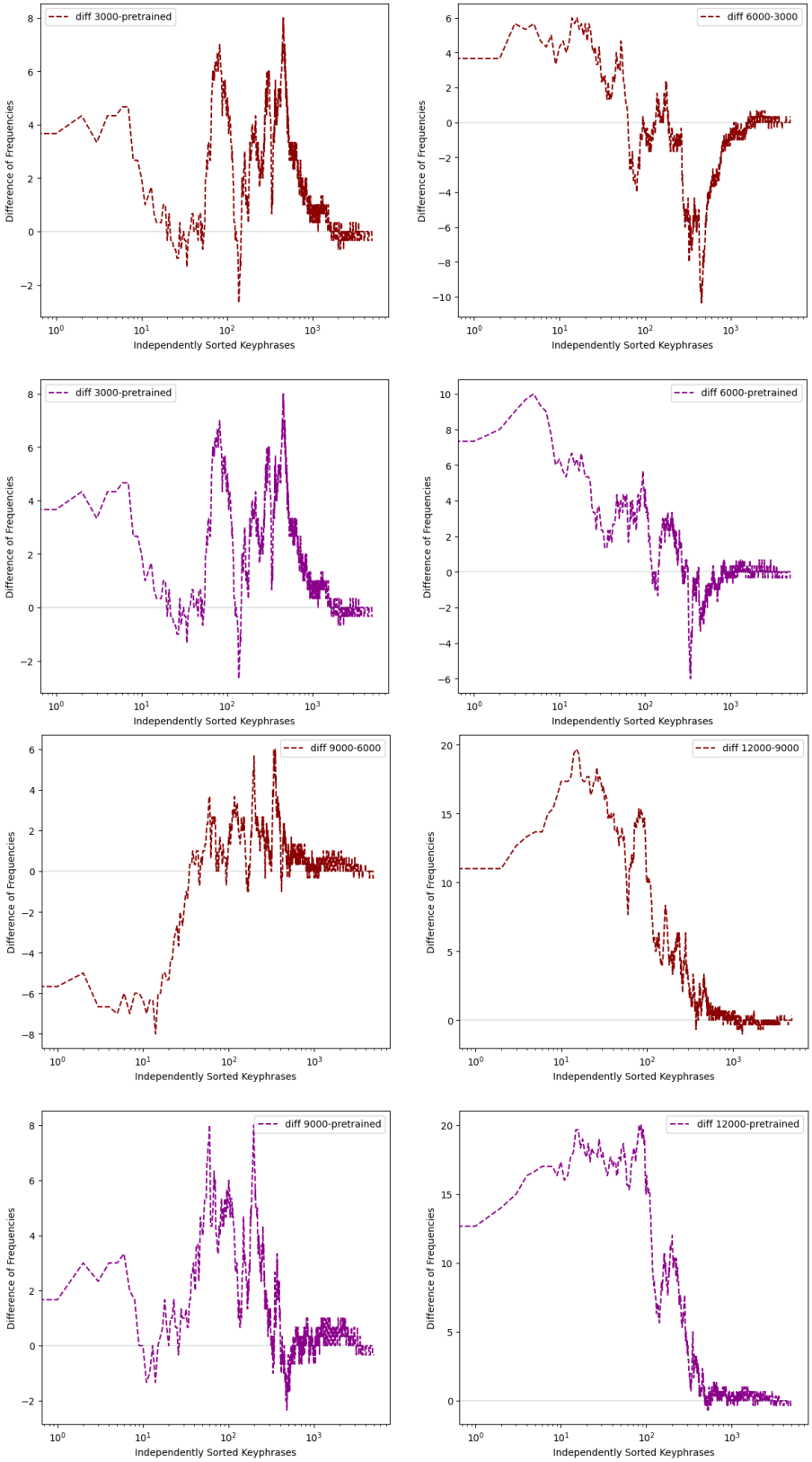


Figure 12: *Difference in the frequencies of keyphrases with K=4 and nHops=4 and cut-off=200*

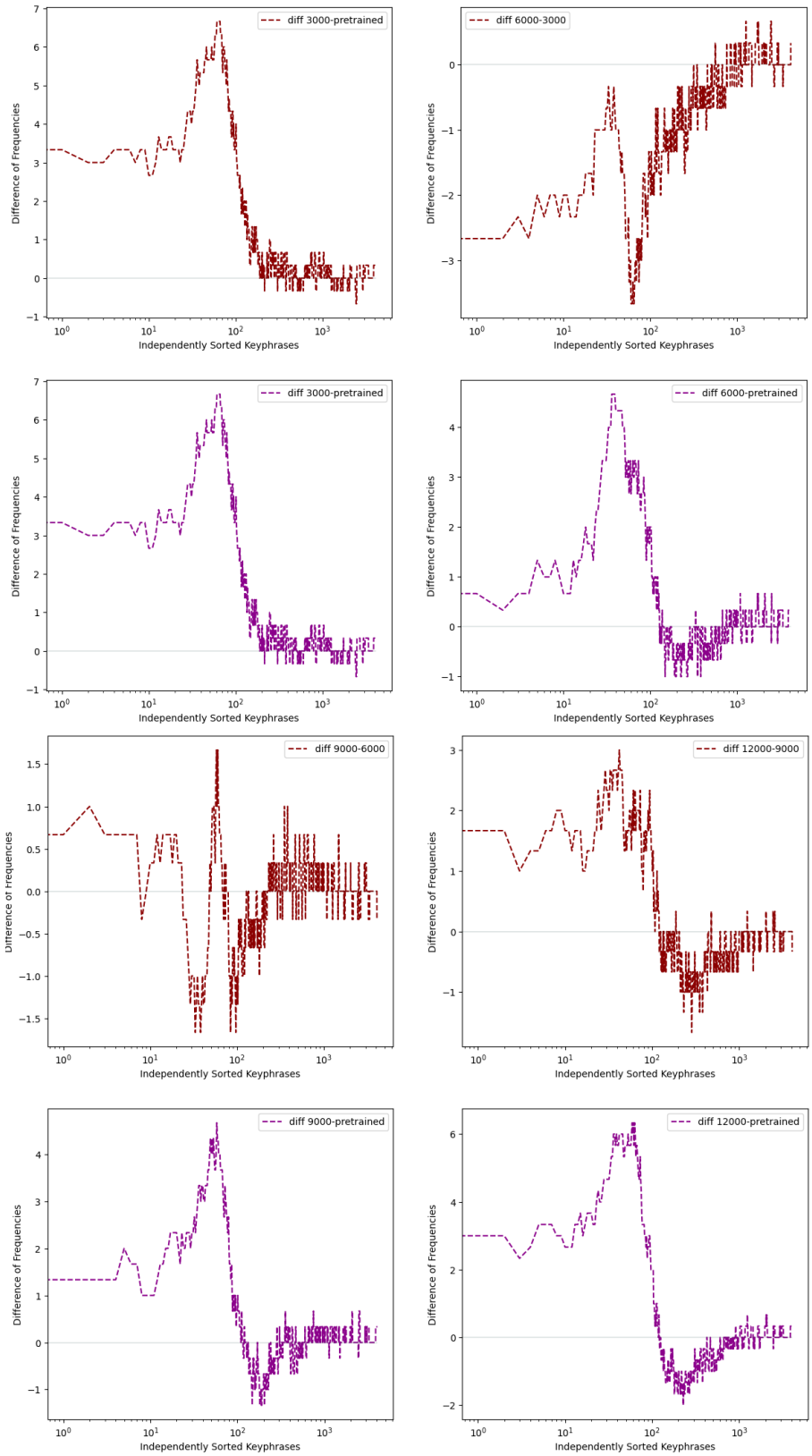


Figure 13: *Difference in the frequencies of keyphrases with K=3 and nHops=3 and cut-off=200*

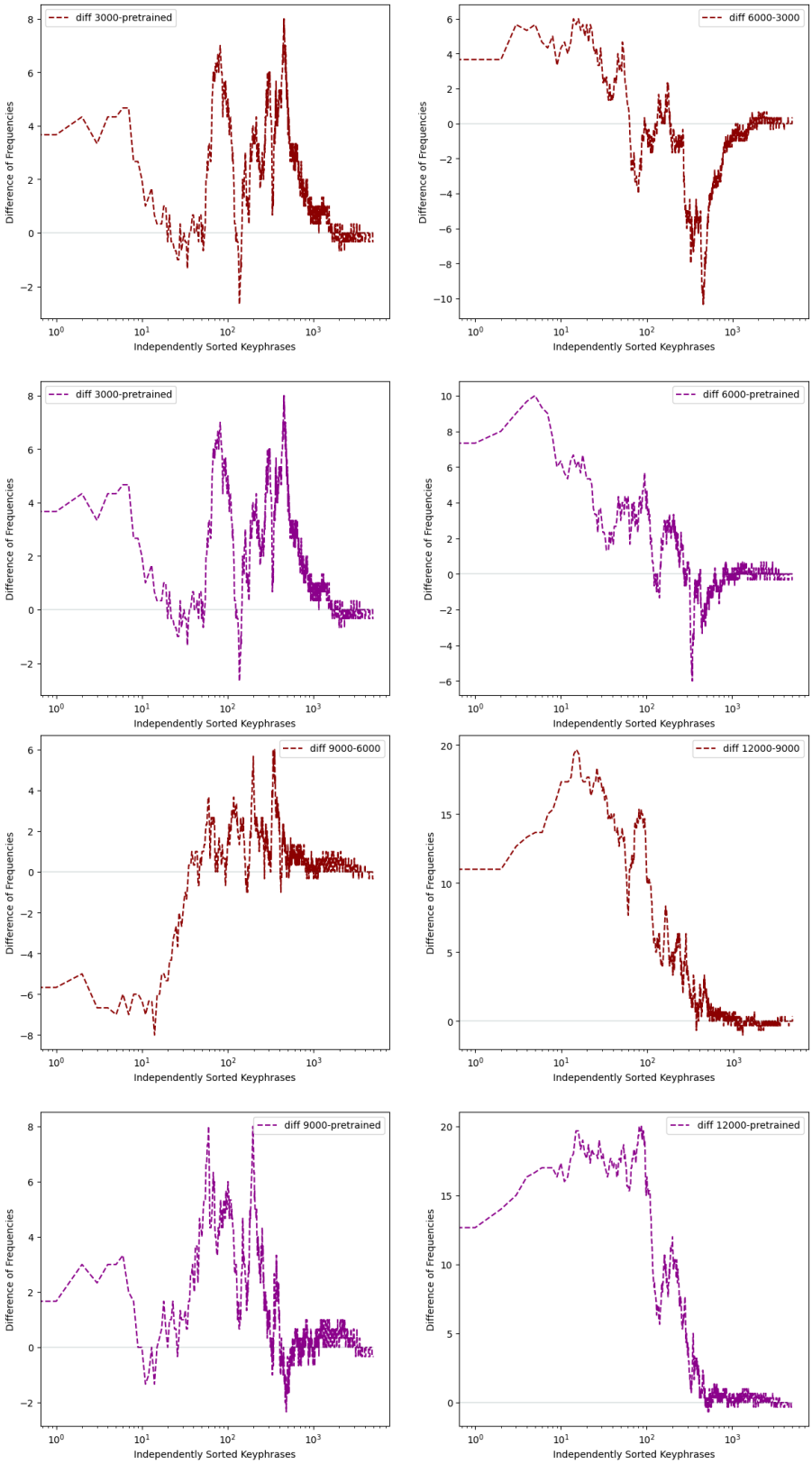


Figure 14: *Difference in the frequencies of keyphrases with K=3 and nHops=4 and cut-off=200*

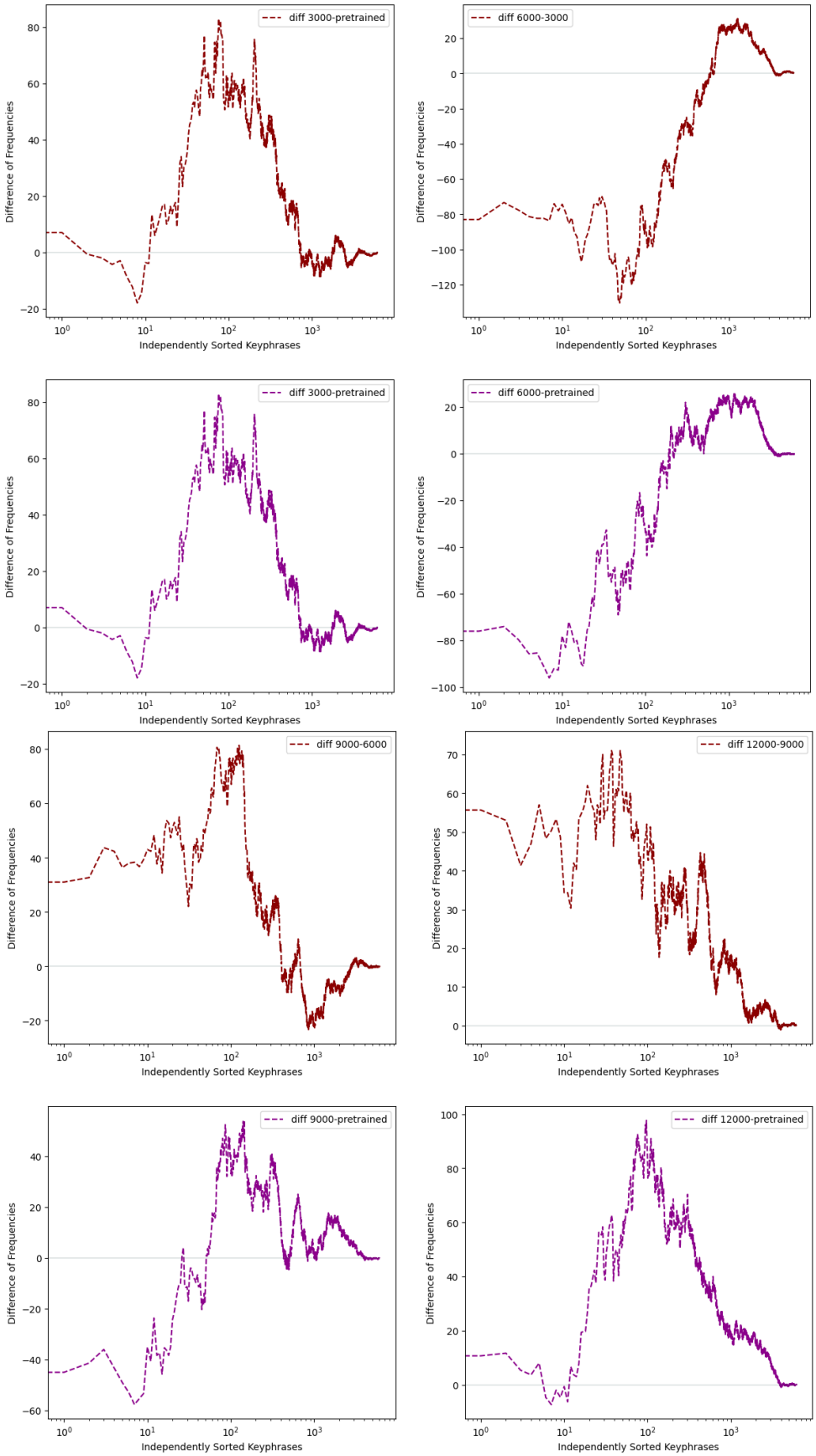
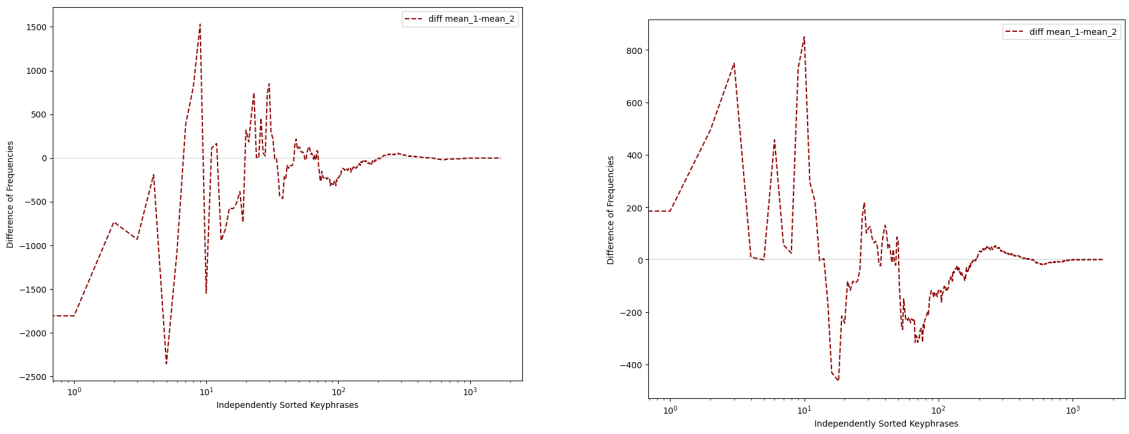


Figure 15: *Difference in the frequencies of keyphrases with K=5 and nHops=5 and cut-off=500*

7.1.2 *Extra Experiments*

To verify the pattern identified in the previous section across different models and datasets, this section presents the results of keyphrase analysis for the two German datasets, xnli and x_stance. Figures 16 and 17 display the respective results for xnli and x_stance. The recursive KNN graphs for these analyses are constructed with K=3 and nHops=4. For the xnli dataset, the first model, mean_1, is the superior model, while for x_stance, the second model, mean_2, achieves a higher score. The patterns observed are consistent with previous experiments, showing higher frequencies of important keyphrases in better-performing models. These results demonstrate the efficacy of keyphrase analysis in comparing different models and correlating them with model performance.

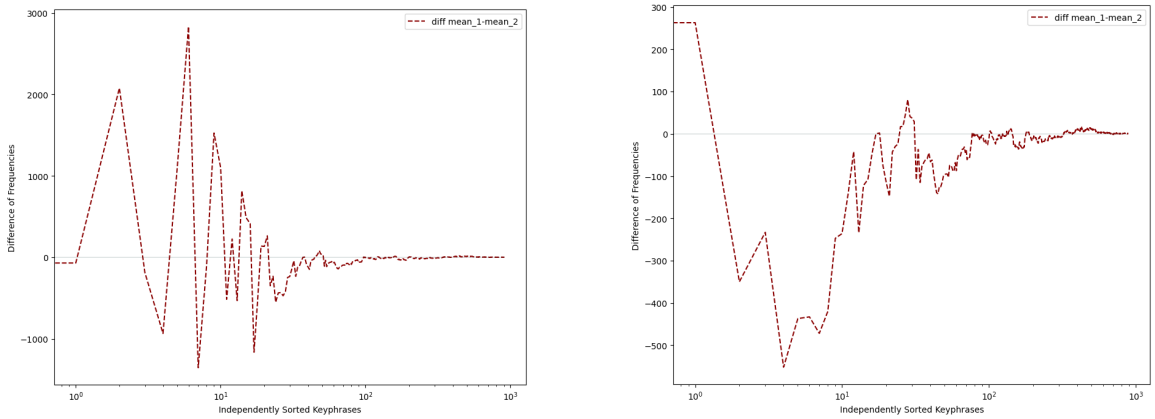


(a) *Frequencies of all keyphrases*

(b) *Frequencies of keyphrases with cut-off=20*

Figure 16: *Distributions of xnli dataset*

Other features of the recursive KNN graphs, such as mean cosine similarities and frequencies of keyphrases within KNN environments, are extracted. Figure 18 illustrates the distribution of environments based on keyphrases and mean cosine similarities for the xnli dataset. The corresponding figure for the x_stance is provided in the Appendix (23). These results highlight the effectiveness of different features of recursive KNN graphs and their structure in distinguishing between different models and analyzing model performance. There appears to be a relevant relationship between the models' performances on the datasets and particularly the mean cosine similarity. Consequently, these findings serve as motivation for this thesis to further research into recursive KNN graphs for evaluating language models without ground truth.



(a) *Frequencies of all keyphrases*

(b) *Frequencies of keyphrases with cut-off=20*

Figure 17: *Distributions of x_stance dataset*

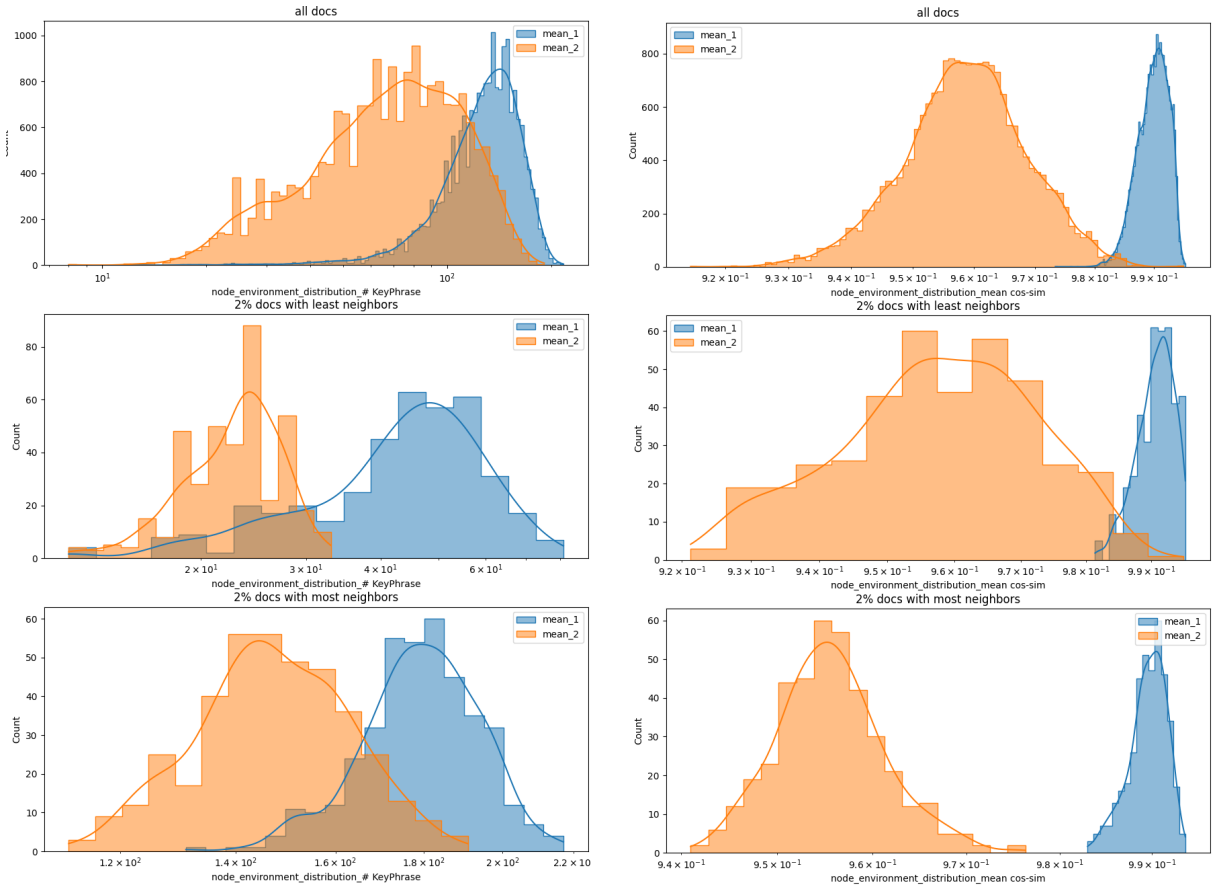


Figure 18: *Properties of xnli's recursive KNN graph*
Left: Distribution of the environments based on the frequencies of keyphrases. Right: Distribution of the environments based on the mean cosine similarity.

7.2 DIFFERENT DATASETS ON THE SAME MODEL

This section presents and evaluates the performance of keyphrase analysis applied to various datasets using the same model, specifically the saved checkpoint at step 12000. The datasets—AmazonCats, MedQuad, FEVER, and Emotional-Reddit—span different domains, and their embeddings were generated using the aforementioned checkpoint. Due to varying numbers of words and document lengths across these datasets, normalization is necessary for effective comparison. The normalization process involves calculating the total number of words separated by white space in each selected subset of these datasets. MedQuad contains the largest word count at 3,438,383, followed by AmazonCats with 2,083,492, Emotional-Reddit with 251,161, and FEVER with 161,823 words. Utilizing these word counts, all datasets are scaled relative to MedQuad, which has the highest word count. Figures 19 and 20 display the results of the keyphrase analysis for different configurations before and after scaling. Examination of the scaled plots demonstrates the method's effectiveness in detecting domain shifts. Moreover, considering the datasets' properties, FEVER and Emotional-Reddit documents tend to contain common phrases and everyday language, leading to the extraction of more frequent and repetitive keyphrases. Consequently, the frequencies of these keyphrases are higher when normalized compared to the specialized terms found in documents from MedQuad and AmazonCats. This discrepancy is also reflected in the shapes of the frequency distributions. The distributions for FEVER and Emotional-Reddit exhibit similar shapes, containing a sharp decline in frequency after an initial peak of highly frequent phrases, whereas the distributions for MedQuad and AmazonCats show a more uniform decrease in frequency. Hence, this method has demonstrated its utility in examining the characteristics of various datasets.

Normalization based only on word count enables comparison of keyphrase frequencies across diverse datasets. However, by multiplying the importance scores of keyphrases with their frequencies, one can compute their weighted frequencies. This method of normalization considers both commonly occurring keyphrases with low importance scores, such as those found in FEVER and Emotional-Reddit, and rarer keyphrases with high importance scores, like those in MedQuad and AmazonCats. Given that the frequency of co-occurring phrases is normalized by each phrase's occurrence count for the computation of the importance scores, additional scaling of weighted frequencies by the word count is unnecessary. The outcomes of these weighted frequencies are demonstrated in the figure 21. Considering the Longformer model that is fine-tuned on the AmazonCats, there is a correlation between model performance and the distribution of weighted frequencies, with higher scores (table 4) and weighted frequencies for MedQuad and AmazonCats than the other two datasets. This suggests that keyphrases of higher importance positively influence document clustering in the creation of

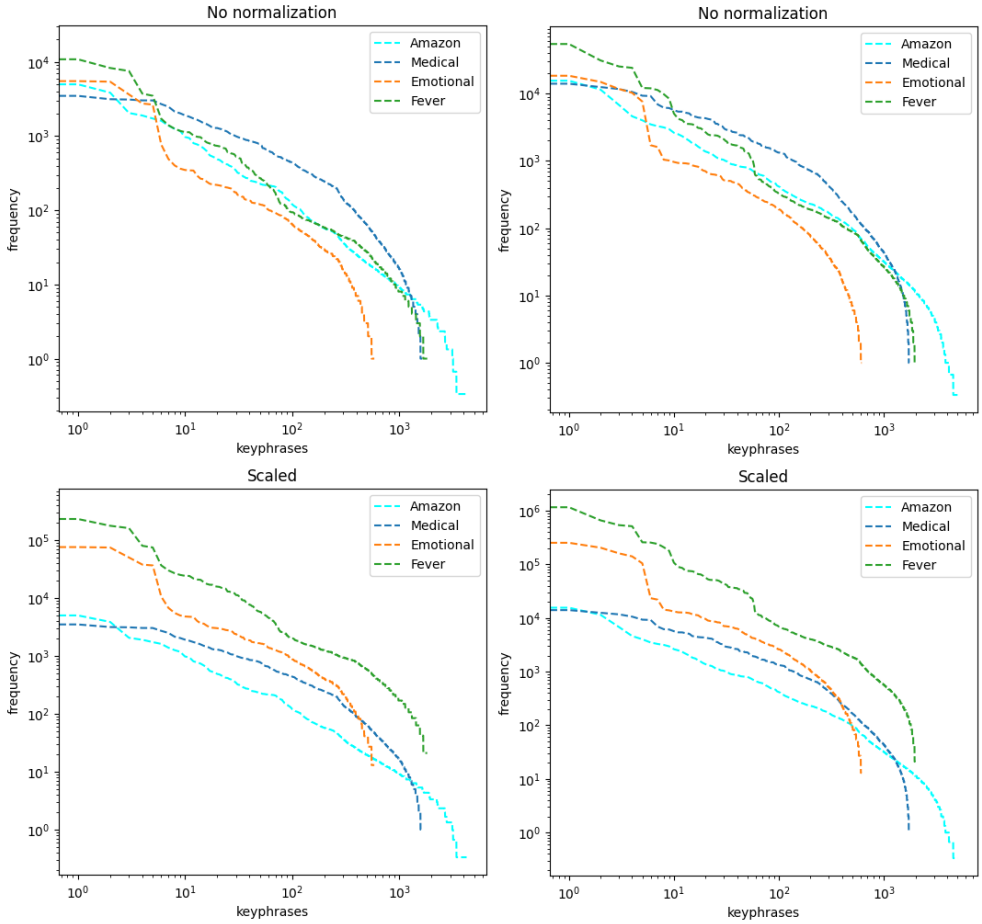


Figure 19: *Frequency distributions of different datasets*
Left: K=3 and nHops=3. Right: K=3 and nHops=4
The overall shapes of each dataset's distribution remain similar across different configurations, with variations primarily happening in the frequencies of the keyphrases.

recursive KNN graphs and also correlate with the tokenizer's fertility, where more specialized keyphrases result in higher fertility. Additionally, AmazonCats exhibits the highest weighted frequencies compared to other datasets, which are considered out-of-distribution for this fine-tuned model. Therefore, the keyphrase analysis method demonstrates potential in identifying domain shifts and out-of-distribution datasets. It has also shown promising results in analyzing the performance of the models. Further research could explore its application in evaluating different language models on datasets lacking ground truth and in downstream tasks.

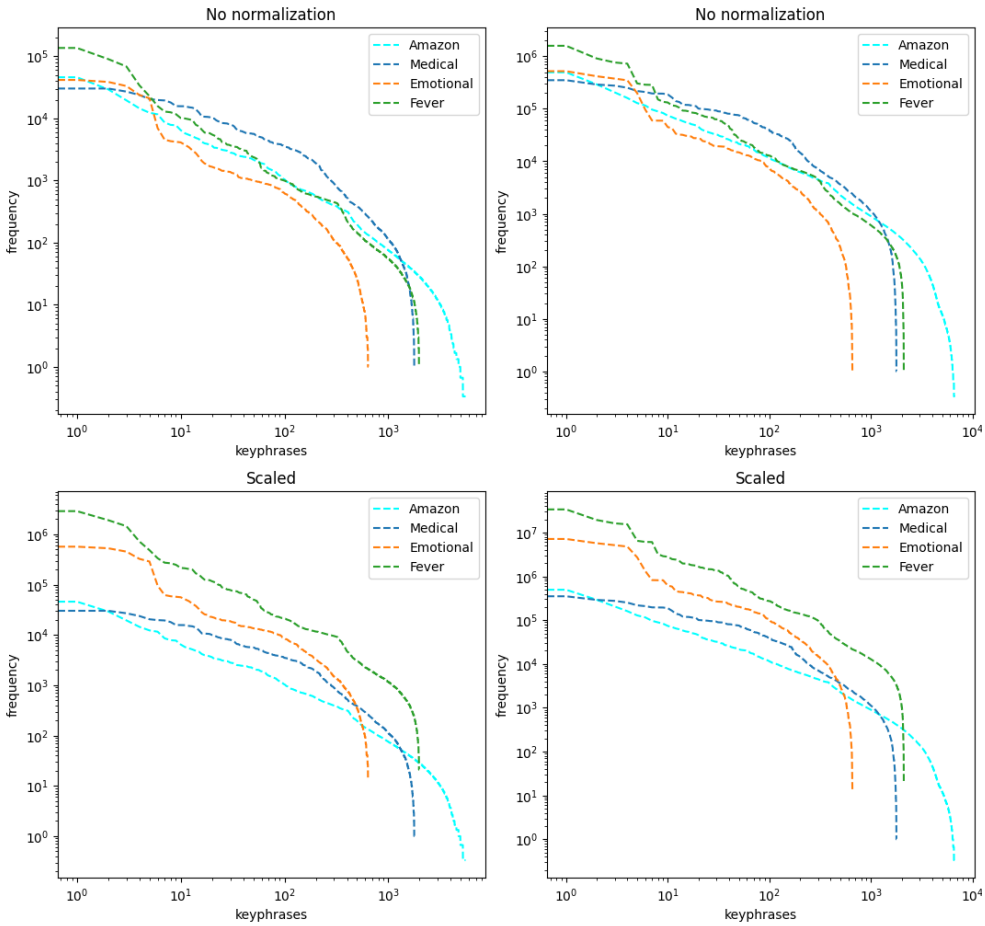


Figure 20: *Frequency distributions of different dataset*
Left: K=4 and nHops=4. Right: K=5 and nHops=5
Because of the diverse range of products described in the AmazonCats dataset, it covers a wide range of topics. Consequently, it contains a larger number of specialized keyphrases compared to the other datasets after the filtering process.

Table 4: *Model's performance on different datasets*

Dataset	Accuracy	Loss
AmazonCats-13k	0.56	2.66
MedQuad	0.62	2.38
Emotional-Reddit	0.46	3.16
FEVER	0.4	4.24

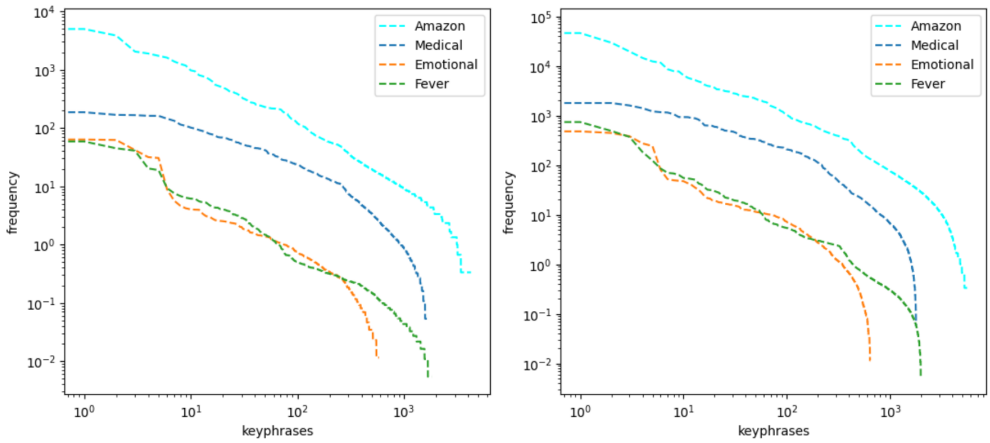


Figure 21: *Weighted frequency distributions of keyphrases for different datasets*

8

CONCLUSION

This thesis tackles the challenge of evaluating and analyzing language models in the absence of ground truth data. The evaluation methodology, employing KNN graphs and keyphrase analysis, has demonstrated significant effectiveness in analyzing out-of-distribution datasets and assessing natural language model performance. This approach not only facilitated a deeper understanding of the embedding spaces generated by different models but also demonstrated its utility in distinguishing between models based on their ability to handle diverse linguistic domains. Specifically, a subset of the AmazonCat 13k dataset was utilized to fine-tune and train a Longformer model. Subsequently, the embeddings from the evaluation set were generated using saved checkpoints to construct recursive KNN graphs and to compare these checkpoints through the frequency distributions of the keyphrases extracted. The findings revealed that superior models exhibit higher frequencies of important and specialized keyphrases compared to earlier checkpoints. This pattern was also observed in additional experiments conducted with the xnli and x_stance datasets.

Furthermore, diverse datasets from various domains were selected based on the tokenizer's fertility. These included MedQuad, FEVER, and Emotional-Reddit. Employing the most recent checkpoint, embeddings for these datasets were also computed, and frequency distributions of the keyphrases were derived from the generated graphs. Comparison of these normalized distributions, relative to the total word count, demonstrated the method's capacity to differentiate between datasets and domains effectively. Additionally, these distributions offered profound insights into the characteristics of each dataset, indicating whether the dataset documents contained more commonplace and simpler words and phrases, or more specialized terminology. This observation led to the multiplication of the importance scores of keyphrases by their initial frequencies to calculate their weighted frequencies. Since the frequency of co-occurring phrases is normalized by each phrase's occurrence count during the computation of importance scores, it negates the need for further scaling of weighted frequencies by the word count. This normalization technique accounts for both frequently occurring keyphrases with low importance scores and rarer keyphrases with high importance scores. The results underscore the method's effectiveness in identifying

out-of-distribution datasets, as evidenced by the distribution of AmazonCats, the dataset used for fine-tuning, which was higher than the others. Additionally, there was a correlation between the distributions and the model's performance, with the model scoring higher for AmazonCat and MedQuad compared to FEVER and Emotional-Reddit.

The results of this thesis have demonstrated that the recursive KNN graphs with keyphrase analysis hold promising potential for the intended tasks of analyzing foundation model performance and detecting out-of-distribution sets. Therefore, they can serve as foundational steps for evaluating models in the absence of ground truth and warrant further exploration in future research endeavors.

8.1 LIMITATION

The method presented in this thesis has certain limitations at its current stage, as discussed in this section. One such limitation concerns the determination of the correct cut-off value for difference plots in section 7.1. This value may vary across different models, datasets, and configurations, needing further research to ascertain optimal cutoff values for specific models and datasets. Additionally, the absence of a numerical score that quantifies the quality of the frequency distribution of keyphrases represents another limitation that warrants further attention.

Another constraint of this method is its dependency on the size of the datasets. The methodology requires a substantial number of keyphrases to generate meaningful plots that can yield further insights. Consequently, smaller datasets containing fewer keyphrases do not facilitate adequate analysis. While larger datasets provide more keyphrases and potentially enable better analysis, care must be taken to avoid excessively large datasets, which can slow down the process due to the high number of documents involved.

8.2 FUTURE WORK

Future research could focus on conducting experiments with various models and datasets to examine the correlation between the keyphrase analysis method and different downstream tasks. Additionally, the effect of different tokenizers could be independently studied. The performance could also be influenced by varying the keyphrase extraction techniques. Therefore, exploring other techniques and algorithms could maybe improve performance beyond that achieved with the Textrank algorithm. Additionally, techniques such as Approximate Nearest Neighbors (ANN) [4] and Nearest Neighbor Descent [18] can be employed to accelerate the construction of KNN graphs and reduce memory usage.

To determine optimal cut-off values for difference plots and enhance normalization techniques for domain shift plots, investigations could explore the relationship between dataset size and extracted keyphrases and the importance scores assigned by keyphrase extraction methods.

Ultimately, developing a numerical score to quantify the quality of keyphrase frequency distribution and its correlation with model performance would be beneficial. This would aid in the analysis of this method and the evaluation of different language models.

BIBLIOGRAPHY

[1] Rakesh Agrawal and Ramakrishnan Srikant. "Fast algorithms for mining association rules in large databases". In: *Proceedings of the 20th International Conference on Very Large Data Bases*. VLDB Endowment. 1994, pp. 487–499.

[2] Explosion AI. *spacy-models release de_core_news_sm-3.7.0*. https://github.com/explosion/spacy-models/releases/tag/de_core_news_sm-3.7.0. Accessed: 2024-02-01. YYYY.

[3] Explosion AI. *spacy-models release en_core_web_sm-3.7.1*. https://github.com/explosion/spacy-models/releases/tag/en_core_web_sm-3.7.1. Accessed: 2024-02-01. YYYY.

[4] Alexandr Andoni, Piotr Indyk, and Ilya P. Razenshteyn. "Approximate Nearest Neighbor Search in High Dimensions". In: *ArXiv* abs/1806.09823 (2018). URL: https://api.semanticscholar.org/CorpusID:49429729.

[5] Iz Beltagy, Matthew E. Peters, and Arman Cohan. "Longformer: The Long-Document Transformer". In: *arXiv:2004.05150* (2020).

[6] Asma Ben Abacha and Dina Demner-Fushman. "A Question-Entailment Approach to Question Answering". In: *BMC Bioinform.* 20.1 (2019), 511:1–511:23. URL: https://bmcbioinformatics.biomedcentral.com/articles/10.1186/s12859-019-3119-4.

[7] K. Bhatia et al. *The extreme classification repository: Multi-label datasets and code*. 2016. URL: http://manikvarma.org/downloads/XC/XMLRepository.html.

[8] David M Blei, Andrew Y Ng, and Michael I Jordan. "Latent dirichlet allocation". In: *Journal of machine Learning research* 3.Jan (2003), pp. 993–1022.

[9] Florian Boudin. "pke: an open source python-based keyphrase extraction toolkit". In: *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: System Demonstrations*. Osaka, Japan, Dec. 2016, pp. 69–73. URL: http://aclweb.org/anthology/C16-2015.

[10] Carlo Abi Chahine et al. "Context and Keyword Extraction in Plain Text Using a Graph Representation". In: *CoRR* abs/0912.1421 (2009). arXiv: 0912.1421. URL: http://arxiv.org/abs/0912.1421.

[11] Yu-Chu Chang et al. "A Survey on Evaluation of Large Language Models". In: *ArXiv* abs/2307.03109 (2023). URL: https://api.semanticscholar.org/CorpusID:259360395.

[12] *Chatgpt*. https://icy-flower-0734d3310.2.azurestaticapps.net/blog/chatgpt. Accessed on November 30, 2022. 2022.

[13] Sishuo Chen et al. "Holistic Sentence Embeddings for Better Out-of-Distribution Detection". In: *ArXiv* abs/2210.07485 (2022). URL: https://api.semanticscholar.org/CorpusID:252907352.

[14] Alexis Conneau et al. "XNLI: Evaluating Cross-lingual Sentence Representations". In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018.

[15] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *CoRR* abs/1810.04805 (2018). arXiv: 1810.04805. URL: http://arxiv.org/abs/1810.04805.

[16] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 2019, pp. 4171–4186.

[17] Amit Dhurandhar et al. "Ranking Large Language Models without Ground Truth". In: *ArXiv* abs/2402.14860 (2024). URL: https://api.semanticscholar.org/CorpusID:267898009.

[18] Wei Dong, Moses Charikar, and K. Li. "Efficient k-nearest neighbor graph construction for generic similarity measures". In: *The Web Conference*. 2011. URL: https://api.semanticscholar.org/CorpusID:207186093.

[19] D. Eppstein, M.S. Paterson, and F.F Yao. "On Nearest-Neighbor Graphs". In: *Discrete Comput Geom 17*. Springer. 1997, pp. 263–282.

[20] Behnam Fanitabasi and Jörg Kindermann. *GitHub*. https://github.com/behnamfani/KNN-diagnosis-tool. 2023.

[21] Wikimedia Foundation. *Wikimedia Downloads*. URL: https://dumps.wikimedia.org.

[22] *GitHub*. https://github.com/behnamfani/KNN-diagnosis-tool. 2023.

[23] *GPT-3.5*. https://platform.openai.com/docs/model-index-for-researchers. Accessed on February 10, 2023. 2022.

[24] Jiawei Han, Jian Pei, and Yiwen Yin. "Mining frequent patterns without candidate generation: A frequent-pattern tree approach". In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. ACM. 2000, pp. 1–12.

[25] Hamel Husain et al. "CodeSearchNet challenge: Evaluating the state of semantic code search". In: *arXiv preprint arXiv:1909.09436* (2019).

[26] Xinzhe Li et al. *A Survey on Out-of-Distribution Evaluation of Neural NLP Models*. 2023. arXiv: 2306.15261 [cs.CL].

[27] Yinhan Liu et al. "RoBERTa: A Robustly Optimized BERT Pretraining Approach". In: *ArXiv* abs/1907.11692 (2019). URL: https://api.semanticscholar.org/CorpusID:198953378.

[28] Malteos. *GPT2-XL German*. https://huggingface.co/malteos/gpt2-xl-wechsel-german. 2022.

[29] *Masked Language Modeling Script*. https://github.com/huggingface/transformers/blob/main/examples/pytorch/language-modeling/run_mlm.py. Accessed: 2024-05-01.

[30] Rada Mihalcea and Paul Tarau. "TextRank: Bringing Order into Text". In: *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*. Barcelona, Spain: Association for Computational Linguistics, July 2004, pp. 404–411. URL: https://aclanthology.org/W04-3252.

[31] Tomas Mikolov et al. "Efficient estimation of word representations in vector space". In: *arXiv preprint arXiv:1301.3781* (2013).

[32] Malte Ostendorff and Georg Rehm. *Efficient Language Model Training through Cross-Lingual and Progressive Transfer Learning*. 2023. DOI: 10.48550/ARXIV.2301.09626.

[33] Lawrence Page et al. "The PageRank Citation Ranking : Bringing Order to the Web". In: *The Web Conference*. 1999. URL: https://api.semanticscholar.org/CorpusID:1508503.

[34] Kishore Papineni et al. "Bleu: a Method for Automatic Evaluation of Machine Translation". In: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Ed. by Pierre Isabelle, Eugene Charniak, and Dekang Lin. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, July 2002, pp. 311–318. DOI: 10.3115/1073083.1073135. URL: https://aclanthology.org/P02-1040.

[35] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[36] Jeffrey Pennington, Richard Socher, and Christopher D Manning. "GloVe: Global Vectors for Word Representation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1532–1543.

[37] Andres Pollano, Anupam Chaudhuri, and Anj Simmons. "Detecting out-of-distribution text using topological features of transformer-based language models". In: *ArXiv* abs/2311.13102 (2023). URL: https://api.semanticscholar.org/CorpusID:265351715.

[38] Alec Radford and Karthik Narasimhan. "Improving Language Understanding by Generative Pre-Training". In: 2018. URL: https://api.semanticscholar.org/CorpusID:49313245.

[39] Nils Reimers and Iryna Gurevych. "Sentence-bert: Sentence embeddings using siamese bert-networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 3982–3992.

[40] Stuart Rose et al. "Automatic Keyword Extraction from Individual Documents". In: Mar. 2010, pp. 1–20. ISBN: 9780470689646. DOI: 10.1002/9780470689646.ch1.

[41] Phillip Rust et al. "How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models". In: *ArXiv* abs/2012.15613 (2020). URL: https://api.semanticscholar.org/CorpusID:229924220.

[42] Rico Sennrich, Barry Haddow, and Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units". In: *ArXiv* abs/1508.07909 (2015). URL: https://api.semanticscholar.org/CorpusID:1114678.

[43] Ashish Sharma et al. "A Computational Approach to Understanding Empathy Expressed in Text-Based Mental Health Support". In: *EMNLP*. 2020.

[44] Oleh Shliazhko et al. *mGPT: Few-Shot Learners Go Multilingual*. 2022. DOI: 10.48550/ARXIV.2204.07580. URL: https://arxiv.org/abs/2204.07580.

[45] Phillip Benjamin Strobel et al. "Evaluation of HTR models without Ground Truth Material". In: *International Conference on Language Resources and Evaluation*. 2022. URL: https://api.semanticscholar.org/CorpusID:246015839.

[46] James Thorne et al. "FEVER: a Large-scale Dataset for Fact Extraction and VERification". In: *NAACL-HLT*. 2018.

[47] *Tokenizers Documentation*. https://huggingface.co/docs/tokenizers/en/index. Accessed: 2024-05-01.

[48] *Transformers Documentation*. https://huggingface.co/docs/transformers/en/index. Accessed: 2024-05-01.

[49] Jannis Vamvas and Rico Sennrich. "X -stance: A Multilingual Multi-Target Dataset for Stance Detection". In: *ArXiv* abs/2003.08385 (2020). URL: https://api.semanticscholar.org/CorpusID:212747834.

[50] Ashish Vaswani et al. "Attention is All you Need". In: *Neural Information Processing Systems*. 2017. URL: https://api.semanticscholar.org/CorpusID:13756489.

[51] Yukun Zhu et al. "Aligning Books and Movies: Towards Story-Like Visual Explanations by Watching Movies and Reading Books". In: *The IEEE International Conference on Computer Vision (ICCV)*. Dec. 2015.

LIST OF ABBREVIATIONS

BERT	Bidirectional Encoder Representations from Transformers
BPE	Byte-Pair Encoding
CNN	Convolutional Neural Network
XMTC	extreme multi-label text classification
HTR	Handwritten Text Recognition
KNN	K-Nearest Neighbour
LDA	Latent Dirichlet Allocation
LM	Language Model
LLM	Large Language Model
ML	Machine Learning
MLM	Masked Language Modeling
NaN	Not a Number
NLI	Natural Language Inference
NLP	Natural Language Processing
OOD	Out-of-Distribution
PP	Perplexity
RNN	Recurrent Neural Network
TF-IDF	Term Frequency-Inverse Document Frequency

A

APPENDIX

This section has other outcomes and figures of the experiments.

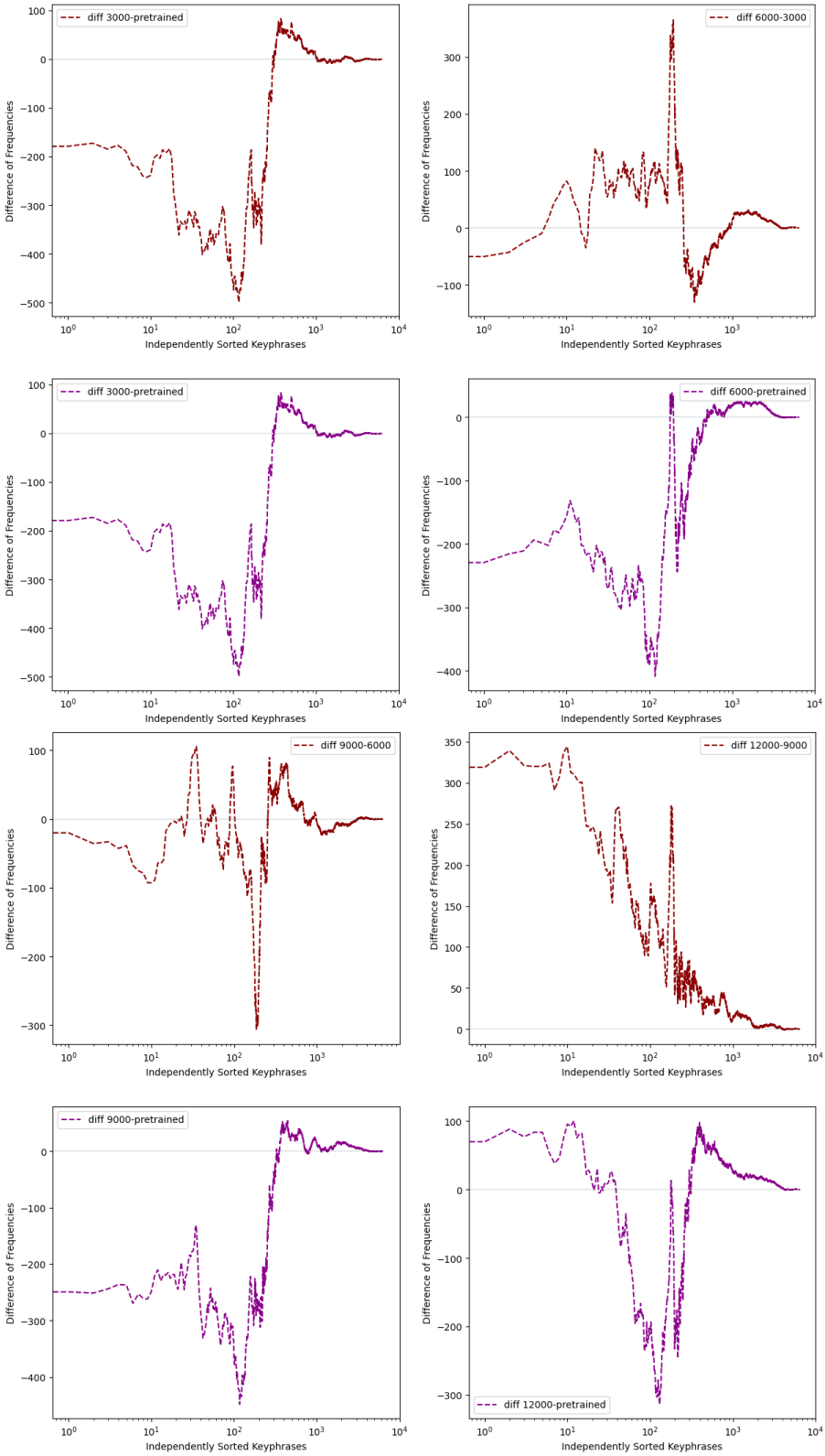


Figure 22: *Difference in the frequencies of keyphrases with K=5 and nHops=5 and cut-off=200*

One challenge encountered in generating plots depicting differences lies in determining the appropriate cut-off number for enhanced analysis as it can be varied with different models, datasets, and configurations. Further research is needed to investigate and establish optimal cut-off values for different models and datasets.

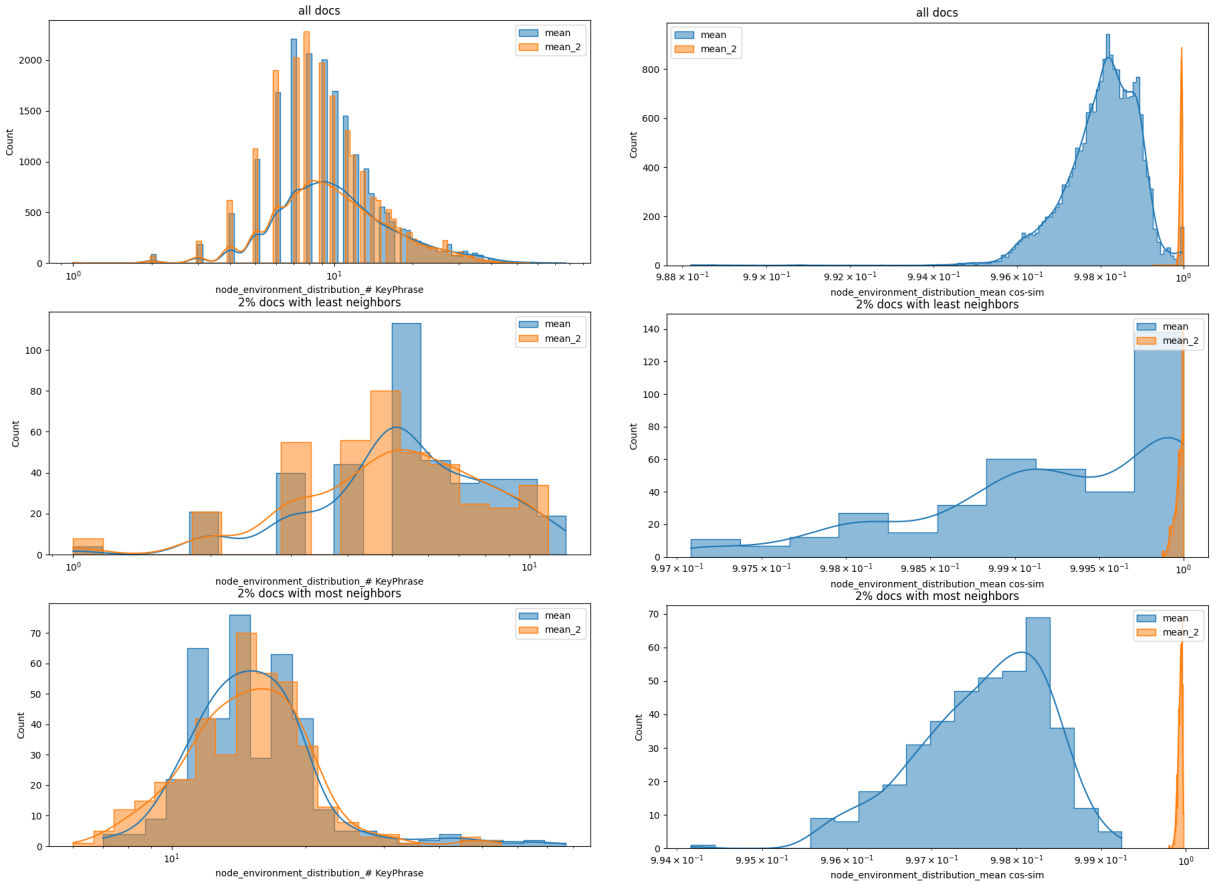


Figure 23: *Properties of x_stance's recursive KNN graph*
Left: Distribution of the environments based on the frequencies of keyphrases. Right: Distribution of the environments based on the mean cosine similarity. The number of keyphrases in the dataset decreased significantly from 20,000 to 1299 after undergoing the filtering steps. This reduction left too few keyphrases for achieving significant results, highlighting a limitation of the analysis procedure when compared to the cos-sin similarity method discussed in this section.